Appunti di Machine Learning

Tommaso Signorini

Indice

1 Introduzione e Tipologie di Apprendimento **2**
1.1 Classificazione dei Problemi di Aprendimento 2

2 Formulazione Teorica del Problema di Predizione **2**
2.1 Funzione di Costo (Loss Function) 2

3 Rischio Atteso e Rischio Empirico **3**
3.1 Rischio Atteso 3
3.2 Rischio Empirico e Principio ERM 3

4 Compromesso Bias-Varianza, Overfitting e Underfitting **3**
4.1 Overfitting e Restrizione dello Spazio delle Ipotesi 3
4.2 Analisi del Compromesso 4

5 Regressione Lineare e Derivazione Analitica **4**
5.1 Rischio Empirico per la Regressione Lineare 4
5.2 Derivazione delle Equazioni Normali 5

6 Valutazione della Qualità e Generalizzazione **5**
6.1 Suddivisione Casuale del Dataset 5
6.2 Stima del Rischio Atteso 5

7 L'Ipotesi Bayesiana e il Rischio Bayesiano **6**
7.1 Derivazione dell'Ipotesi Bayesiana per il Costo Quadratico 6

8 Decomposizione Bias-Varianza e Stima-Approssimazione **7**
8.1 Decomposizione Bias-Varianza (Caso Quadratico) 7
8.2 Decomposizione Stima-Approssimazione (Caso Generale) 7

9 Modelli di Base e Metriche di Valutazione **8**
9.1 Ipotesi di Base e Rischio Empirico 8
9.2 Il Coefficiente di Determinazione (R^2) 8

10 Ingegnerizzazione delle Feature (Feature Engineering) **8**
10.1 Codifica delle Variabili Qualitative 9
10.2 Effetti Congiunti e Modelli Non Lineari 9

11 Scelta della Funzione di Costo e MLE **9**
11.1 Maximum Likelihood Estimation (MLE) e Derivazione OLS 9
11.2 LAD (Scarto Assoluto) vs OLS (Scarto Quadratico) 10

12 Modelli Non Parametrici: K-Nearest Neighbors (KNN) **11**

13 Argomenti Avanzati **11**
13.1 Pesatura degli Esempi (Instance Weighting) 11
13.2 Regressione Multi-Output 11

14 Il Ruolo dell'Ottimizzazione nel Machine Learning **12**
14.1 Formulazione Matematica 12
14.2 Minimi Globali vs. Minimi Locali 12

15 Metodi di Ricerca di Ordine Zero (Black-Box) **12**
15.1 Grid Search e Randomized Search 12
15.1.1 La Maledizione della Multidimensionalità (*Curse of Dimensionality*) 13
15.2 Randomized Local Search (RLS) 13

16 Classificazione dei Metodi e Sviluppi di Taylor **14**
16.1 Approssimazioni di Taylor (Caso Univariato $N = 1$) 14
16.2 Approssimazioni di Taylor (Caso Multivariato $N \geq 1$) 14

17 Condizioni di Ottimalità e Teoria della Convessità **15**
17.1 Condizioni Necessarie del Primo Ordine 15
17.2 Definizione e Caratterizzazioni delle Funzioni Convesse 15
17.3 Criteri Operativi di Convessità e Dimostrazione per le Norme 16
17.4 Importanza della Convessità nelle Funzioni di Costo 16

18 Algoritmo di Discesa del Gradiente (Gradient Descent) **16**
18.1 Formulazione dell'Algoritmo 17
18.2 Il Tasso di Apprendimento (η) 17

19 Teoremi e Dimostrazioni Rigorose di Convergenza del GD **17**
19.1 Dimostrazione per Funzioni Convesse e Gradienti Limitati 17
19.2 Convergenza per Funzioni β-smooth 20

20 Problematiche del GD e Varianti Avanzate **20**
20.1 Problema 1: Movimento a Zig-Zag 20
20.2 Problema 2: Rallentamento nei Punti Critici e Plateau 20
20.3 Varianti del Primo Ordine 20

21 Metodi del Secondo Ordine e Quasi-Newton **21**
21.1 Il Metodo di Newton-Raphson 21
21.2 Limiti dei Metodi del Secondo Ordine nel ML 21
21.3 Metodi Quasi-Newton (BFGS e L-BFGS) 21

22 Ottimizzazione del Rischio Empirico e Varianti Stocastiche **22**
22.1 Formulazione del Rischio Empirico nel ML 22
22.2 Batch Gradient Descent (GD) 22
22.3 Stochastic Gradient Descent (SGD) 22
22.3.1 Esempio 1: SGD per la Regressione Lineare (Regola LMS) 23
22.3.2 Esempio 2: SGD per la Regressione LAD (Norma L_1) 23
22.4 Mini-Batch SGD 23
22.5 Confronto Sintetico tra le Varianti 23
22.6 Note Implementative in `scikit-learn` 23

23 Partizionamento dei Dati e Selezione degli Iperparametri **24**
23.1 Divisione in Test e Train Set 24
23.2 Selezione degli Iperparametri e Grid Search 24
23.3 Architettura a Tre Insiemi 25
23.4 Dimensionamento dei Set e Teoria della Stima 25
23.4.1 Come si Valuta la Grandezza di Validation e Test Set 26
23.5 Validazione Incrociata (K-Fold Cross-Validation) 27

24 Metodi di Riduzione delle Feature **27**
24.1 Selezione di Sottoinsiemi (Feature Selection) 27
24.1.1 Best Subset Selection 27
24.1.2 Selezione Passo-Passo Sequenziale (Stepwise Selection) 28
24.1.3 Selezione Basata sui Coefficienti 28

25 Metodi di Regolarizzazione (Shrinkage) **28**
25.1 Regressione Ridge (Regolarizzazione L_2) 28
25.1.1 Impatto del Parametro λ sui Coefficienti in Ridge 29
25.2 Regressione LASSO (Regolarizzazione L_1) 29
25.2.1 Impatto di λ nel LASSO 29
25.3 Trade-Off Bias-Varianza ed Evoluzione dei Modelli 30
25.4 Standardizzazione delle Feature 30

26 Sintesi delle Interfacce in Scikit-Learn **30**

27 Introduzione ai Problemi di Classificazione **31**
27.1 Funzione di Costo e Valutazione del Rischio 31

28 Il Metodo K-Nearest Neighbor (K-NN) **32**
28.1 La Nozione di Distanza 32

29 La Scelta di K: Il Compromesso Bias-Varianza **32**

30 Importanza della Feature Selection **33**

31 Efficienza Computazionale **33**

32 Garanzie Teoriche e Maledizione della Dimensionalità **34**

33 Introduzione alla Classificazione Generativa **34**
33.1 Il Criterio Bayesiano e la Minimizzazione del Rischio 35
33.2 Ottimalità del Criterio Bayesiano e Rischio Atteso 35
33.3 Il Discriminante 36

34 Analisi del Discriminante **36**
34.1 Gaussiana Univariata 36
34.2 Derivazione MLE per Gaussiana Univariata 37
34.3 Gaussiana Multivariata 37
34.4 QDA e LDA 38

35 Classificatore Naive Bayes **40**
35.1 Esempio: Classificazione di Testi (Spam / No Spam) 40
35.1.1 Esempio: Classificazione Testuale (Filtro Spam) 41
35.2 L'uso dei logaritmi e l'Underflow Numerico 41
35.3 I "Cigni Neri" e la Regolarizzazione di Laplace 41

36 Stima di Massima Verosimiglianza (MLE) **42**

1 Introduzione e Tipologie di Apprendimento

L'Apprendimento Automatico (*Machine Learning*) fornisce ai calcolatori la capacità di apprendere da dati ed esperienze senza essere esplicitamente programmati per ogni singolo compito.

Definizione 1.1 (Machine Learning - Tom Mitchell, 1997)**.** Un algoritmo apprende da un'esperienza E rispetto a una classe di compiti T e a una misura di prestazione P, se le sue prestazioni nei compiti in T, misurate da P, migliorano con l'esperienza E.

Differiamo tra l'approccio classico (deduttivo), in cui la relazione input-output è definita in modo formale e analitico, e l'approccio induttivo (Machine Learning), in cui la relazione è desunta tramite esempi registrati.

1.1 Classificazione dei Problemi di Aprendimento

- **Apprendimento Supervisionato:** Ogni esempio di addestramento è fornito nella forma coppia input-output (x, y).
  - *Classificazione:* L'output y è qualitativo/categorico (es. distinzione spam/non spam o identificazione di cifre scritte a mano come nel dataset MNIST).
  - *Regressione:* L'output y è quantitativo/numerico (es. stima del salario o predizione dei ricavi dalle vendite in base agli investimenti pubblicitari).
  - *Output Probabilistico:* L'output rappresenta una probabilità $y \in [0, 1]$ (es. stima di transazioni bancarie fraudolente).
- **Apprendimento Non Supervisionato:** L'algoritmo riceve solo ingressi x senza etichette y, identificando strutture o pattern nei dati (*Clustering*, Riduzione della dimensionalità).
- **Apprendimento per Rinforzo:** Basato su un agente che interagisce dinamica con un ambiente compiendo azioni e ricevendo ricompense.

2 Formulazione Teorica del Problema di Predizione

Sia $\mathcal{X}$ lo **spazio degli ingressi** (es. $\mathcal{X} = \mathbb{R}^3_+$ per investimenti in TV, radio, giornali) e $\mathcal{Y}_0$ lo **spazio delle etichette** (es. $\mathcal{Y}_0 = \mathbb{R}$ per la quantità di prodotto venduta).

L'obiettivo è determinare una **funzione ipotesi** (o regola di predizione) $h : \mathcal{X} \to \mathcal{Y}$ che approssimi in modo accurato la relazione reale tra ingressi e uscite.

2.1 Funzione di Costo (Loss Function)

Per quantificare l'errore commesso dalla funzione ipotesi h su un singolo esempio (x, y), si definisce una funzione di costo $l : \mathcal{Y} \times \mathcal{Y}_0 \to \mathbb{R}_+$, con $l(\hat{y}, y) \geq 0$, dove $\hat{y} = h(x)$.

- **Errore Quadratico (per regressione):**

$$l(\hat{y}, y) \triangleq (\hat{y} - y)^2 \tag{1}$$

- **Funzione di Costo 0-1 (per classificazione):**

$$l(\hat{y}, y) \triangleq \begin{cases} 0 & \text{se } \hat{y} = y \\ 1 & \text{se } \hat{y} \neq y \end{cases} \tag{2}$$

3 Rischio Atteso e Rischio Empirico

3.1 Rischio Atteso

Assumiamo che le coppie (x, y) siano generate in modo indipendente e identicamente distribuito (i.i.d.) da una distribuzione di probabilità $\mathcal{D}$ su $\mathcal{X} \times \mathcal{Y}_0$. Poiché la distribuzione $\mathcal{D}$ rappresenta la vera natura del fenomeno, essa è **ignota**.

Definizione 3.1 (Rischio Atteso)**.** Il rischio atteso $RA(h)$ misura l'errore medio della funzione di predizione h sull'intera distribuzione teorica $\mathcal{D}$:

$$RA(h) \triangleq \mathbb{E}_{(x,y)\sim\mathcal{D}}[l(h(x), y)] \tag{3}$$

In ambito pratico, $**RA(h)$ non è calcolabile analiticamente** poiché $\mathcal{D}$ non è nota. Il problema teorico del Machine Learning consisterebbe nel trovare $h^* = \arg\min_h RA(h)$.

3.2 Rischio Empirico e Principio ERM

In assenza di $\mathcal{D}$, disponiamo di un campione osservato di m esempi, detto **Training Set**:

$$S = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\} \tag{4}$$

In virtù della Legge dei Grandi Numeri, si adotta come surrogato del rischio atteso il **Rischio Empirico**.

Definizione 3.2 (Rischio Empirico)**.** Il rischio empirico $RE_S(h)$ è la media del costo valutata sugli esempi del training set S:

$$RE_S(h) \triangleq \frac{1}{m} \sum_{i=1}^{m} l(h(x^{(i)}), y^{(i)}) \tag{5}$$

Definizione 3.3 (Empirical Risk Minimization - ERM)**.** Il principio ERM impone di selezionare una funzione h che minimizzi il rischio empirico sul dataset di addestramento:

$$\min_{h\in\mathcal{H}} RE_S(h) \tag{6}$$

4 Compromesso Bias-Varianza, Overfitting e Underfitting

Se la minimizzazione del rischio empirico viene condotta sull'insieme di tutte le funzioni possibili ($\mathcal{Y}^{\mathcal{X}}$), si rischia il fenomeno del **sovradattamento (overfitting)**.

4.1 Overfitting e Restrizione dello Spazio delle Ipotesi

Un modello affetto da *overfitting* interpola o memorizza il rumore presente nei dati del training set, ottenendo $RE_S(h) \approx 0$, ma un rischio atteso $RA(h)$ molto elevato per nuovi esempi.

Per evitare l'overfitting, si limita lo spazio delle funzioni ammissibili scegliendo una classe di ipotesi ristretta $\mathcal{H}$ (es. funzioni lineari, polinomi di grado limitato). Questa restrizione introduce un **bias induttivo**, dato dallo scartare tutte le funzioni al di fuori di tale classe.

4.2 Analisi del Compromesso

- **Alta Rigidità (Bias Alto / Underfitting):** Se la classe $\mathcal{H}$ è troppo semplice (es. approssimazione lineare di dati non lineari), la funzione non cattura la struttura dei dati.
- **Alta Flessibilità (Varianza Alta / Overfitting):** Se la classe $\mathcal{H}$ ha troppi parametri, il modello è instabile e richiede un numero m di esempi molto maggiore per generalizzare correttamente.

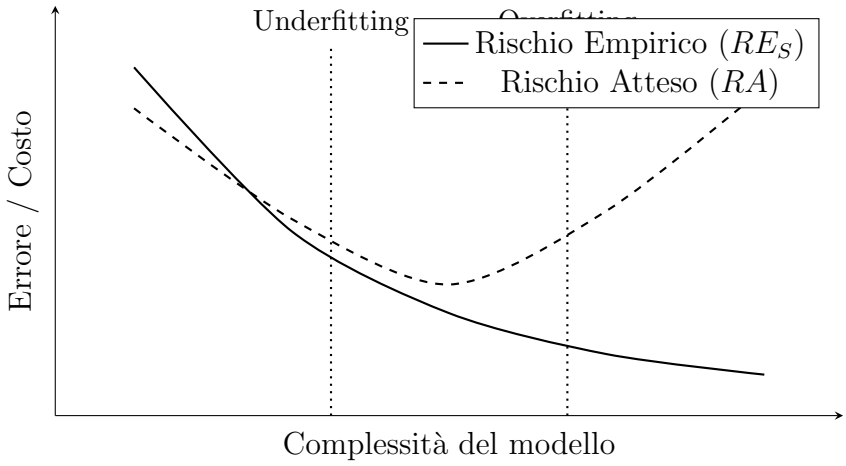


Figura 1: Evoluzione del rischio empirico e atteso al variare della complessità del modello.

5 Regressione Lineare e Derivazione Analitica

Nella regressione lineare, lo spazio delle ipotesi è ristretto alla classe delle funzioni lineari affini $\mathcal{H}_{lin}$:

$$h(x) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \tag{7}$$

dove w_0 rappresenta l'intercetta (*bias*) e $w_1, \ldots, w_d$ sono i pesi dei singoli predittori.

Adottando la convenzione del termine noto esteso $x_0 = 1$, poniamo $x = [1, x_1, \ldots, x_d]^\top \in \mathbb{R}^{d+1}$ e $w = [w_0, w_1, \ldots, w_d]^\top \in \mathbb{R}^{d+1}$, riducendo l'espressione a:

$$h(x) = w^\top x \tag{8}$$

5.1 Rischio Empirico per la Regressione Lineare

Dato un dataset di m esempi rappresentato dalla matrice dei dati $X \in \mathbb{R}^{m \times (d+1)}$ e dal vettore dei target $y \in \mathbb{R}^m$:

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \ldots & x_d^{(1)} \\ 1 & x_1^{(2)} & \ldots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \ldots & x_d^{(m)} \end{bmatrix}, \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix} \tag{9}$$

Il rischio empirico, espresso tramite la funzione di Scarto Quadratico Medio (*MSE*), assume forma matriciale:

$$RE_S(w) = \frac{1}{m}\sum_{i=1}^{m}(w^\top x^{(i)} - y^{(i)})^2 = \frac{1}{m}\|Xw - y\|^2 \tag{10}$$

5.2 Derivazione delle Equazioni Normali

Per minimizzare $RE_S(w)$, poniamo il gradiente rispetto al vettore dei parametri w uguale a zero:

$$\nabla_w RE_S(w) = \frac{2}{m}X^\top(Xw - y) = 0 \tag{11}$$

Da cui si ottiene il sistema delle **Equazioni Normali**:

$$X^\top X w^* = X^\top y \tag{12}$$

Se la matrice $X^\top X$ è invertibile, la soluzione analitica ottimale è data da:

$$w^* = (X^\top X)^{-1}X^\top y \tag{13}$$

6 Valutazione della Qualità e Generalizzazione

Poiché il rischio empirico sul training set $RE_S(h)$ non è un indicatore attendibile della bontà generale del modello (dato invece dal rischi atteso $RA(h)$), la valutazione della qualità va eseguita su un insieme di dati distinto e indipendente, detto **Test Set** T.

6.1 Suddivisione Casuale del Dataset

I dati disponibili vengono suddivisi casualmente per evitare di introdurre bias strutturali:

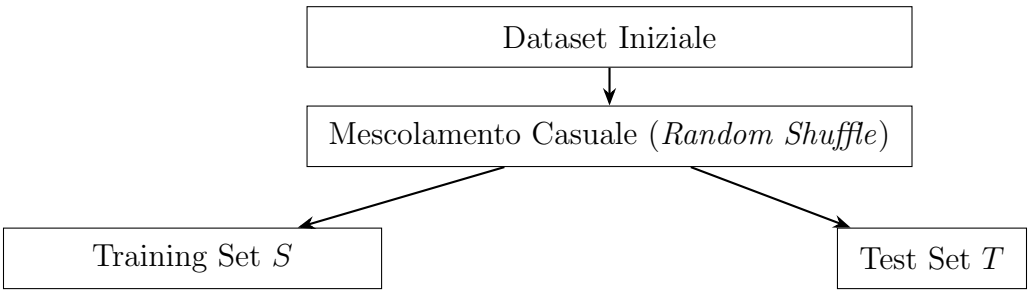


Figura 2: Schema del processo di partizionamento casuale del dataset.

6.2 Stima del Rischio Atteso

Dati $T = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$ esempi di test indipendenti da h:

$$RE_T(h) = \frac{1}{n}\sum_{i=1}^{n} l(h(x^{(i)}), y^{(i)}) \approx RA(h) \tag{14}$$

Poiché T contiene dati mai visti durante la fase di addestramento, il valore $RE_T(h)$ rappresenta una stima non distorta del rischio atteso: $\mathbb{E}_T[RE_T(h)] = RA(h)$.

7 L'Ipotesi Bayesiana e il Rischio Bayesiano

Qual è il minimo rischio atteso teoricamente raggiungibile in assoluto?

Definizione 7.1 (Ipotesi Bayesiana e Rischio Bayesiano)**.** L'**ipotesi Bayesiana** h^* è la funzione che minimizza il rischio atteso tra tutte le funzioni possibili da $\mathcal{X}$ a $\mathcal{Y}$:

$$h^* \triangleq \arg\min_{h:\mathcal{X}\to\mathcal{Y}} RA(h) \tag{15}$$

Il rischio atteso corrispondente $RA(h^*)$ prende il nome di **rischio Bayesiano**.

Il rischio Bayesiano rappresenta il **rumore intrinseco irriducibile** del processo di generazione dei dati. Dipende dall'incertezza stocastica naturale della relazione tra x e y (ad esempio fattori non osservati o errori casuali di misura) e costituisce un limite invalicabile per qualsiasi algoritmo di apprendimento.

7.1 Derivazione dell'Ipotesi Bayesiana per il Costo Quadratico

Consideriamo la regressione con errore quadratico $l(h(x), y) = (h(x) - y)^2$. Esprimendo il rischio atteso tramite la legge delle aspettative condizionate:

$$RA(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(h(x) - y)^2] = \mathbb{E}_x\left[\mathbb{E}_{y|x}[(h(x) - y)^2 \mid x]\right] \tag{16}$$

Per minimizzare $RA(h)$, è sufficiente minimizzare separatamente la quantità interna $\mathbb{E}_{y|x}[(h(x) - y)^2 \mid x]$ per ciascun valore di x. Sviluppando il quadrato:

$$\begin{aligned}\mathbb{E}_{y|x}[(h(x) - y)^2 \mid x] &= \mathbb{E}_{y|x}[h(x)^2 - 2h(x)y + y^2 \mid x] \\ &= h(x)^2 - 2h(x)\mathbb{E}[y \mid x] + \mathbb{E}[y^2 \mid x]\end{aligned} \tag{17}$$

Aggiungendo e sottraendo $(\mathbb{E}[y \mid x])^2$:

$$\begin{aligned}\mathbb{E}_{y|x}[(h(x) - y)^2 \mid x] &= h(x)^2 - 2h(x)\mathbb{E}[y \mid x] + (\mathbb{E}[y \mid x])^2 + \mathbb{E}[y^2 \mid x] - (\mathbb{E}[y \mid x])^2 \\ &= (h(x) - \mathbb{E}[y \mid x])^2 + \mathrm{Var}[y \mid x]\end{aligned} \tag{18}$$

Analizzando i due termini:

- Il secondo termine $\mathrm{Var}[y \mid x]$ rappresenta la varianza di y dato x e **non dipende** dalla scelta di h.
- Il primo termine è sempre non negativo ed è minimizzato (azzerato) ponendo:

$$h^*(x) = \mathbb{E}[y \mid x] \tag{19}$$

L'ipotesi Bayesiana per l'errore quadratico è dunque il **valore atteso condizionato di y dato x**, mentre il rischio Bayesiano è pari alla media della varianza condizionata:

$$RA(h^*) = \mathbb{E}_x[\mathrm{Var}[y \mid x]] \tag{20}$$

8 Decomposizione Bias-Varianza e Stima-Approssimazione

8.1 Decomposizione Bias-Varianza (Caso Quadratico)

Per il modello di regressione con costo quadratico, il rischio atteso di un'ipotesi h appresa si scompone in tre fattori distinti:

Teorema 8.1 (Decomposizione Bias-Varianza)**.**

$$RA(h) = (bias_h)^2 + varianza_h + rischio\ Bayesiano \tag{21}$$

dove:

$$bias_h \triangleq \mathbb{E}[h(x) - h^*(x)] \tag{22}$$

$$varianza_h \triangleq Var[h(x)] = \mathbb{E}\left[(h(x) - \mathbb{E}[h(x)])^2\right] \tag{23}$$

$$rischio\ Bayesiano \triangleq RA(h^*) = \mathbb{E}\left[(h^*(x) - y)^2\right] \tag{24}$$

per vedere questi termini basta scrivere $h(x) - y = h(x) - h^*(x) + h^*(x) - y$ e poi farne il quadrato.

Compromesso Bias-Varianza:

- Modelli più semplici (pochi parametri) hanno un **bias elevato** (*underfitting*), poiché lo spazio $\mathcal{H}$ non contiene funzioni sufficientemente flessibili.
- Modelli più complessi (molti parametri) riducono il bias ma aumentano la **varianza**, diventando molto sensibili allo specifico campione di dati S e richiedendo un numero elevato di esempi per essere stabilizzati.

8.2 Decomposizione Stima-Approssimazione (Caso Generale)

In contesti generali, per qualsiasi problema e spazio di ipotesi $\mathcal{H}$, definiamo:

- $h^* = \arg\min_{h:\mathcal{X}\to\mathcal{Y}} RA(h)$: la migliore funzione teorica assoluta (ipotesi Bayesiana).
- $\bar{h} = \arg\min_{h\in\mathcal{H}} RA(h)$: la migliore funzione appartenente allo spazio di ipotesi $\mathcal{H}$.
- $h \in \mathcal{H}$: l'ipotesi selezionata dall'algoritmo minimizzando il rischio empirico su un campione finito S.

Teorema 8.2 (Decomposizione del Rischio Atteso)**.** *Il rischio atteso dell'ipotesi h si scompone analiticamente in:*

$$RA(h) = \underbrace{\left(RA(h) - RA(\bar{h})\right)}_{Errore\ di\ Stima} + \underbrace{\left(RA(\bar{h}) - RA(h^*)\right)}_{Errore\ di\ Approssimazione} + \underbrace{RA(h^*)}_{Rischio\ Bayesiano} \tag{25}$$

(di fatto abbiamo fatto somme e sottrazioni di quantità uguali).

- **Errore di Approssimazione** ($RA(\bar{h}) - RA(h^*) \geq 0$): misura il costo dovuto alla scelta di uno spazio ristretto $\mathcal{H}$ anziché l'insieme di tutte le funzioni. Diminuisce aumentando la complessità di $\mathcal{H}$.
- **Errore di Stima** ($RA(h) - RA(\bar{h}) \geq 0$): misura l'errore commesso poiché non conosciamo la vera distribuzione $\mathcal{D}$ e dobbiamo stimare la funzione tramite un insieme finito di m esempi S. Dipende dalla "fortuna" nei dati osservati e aumenta se la classe $\mathcal{H}$ è troppo complessa rispetto alla dimensione m del dataset.
- **Rischio Bayesiano** ($RA(h^*)$): rumore stocastico irriducibile del sistema.

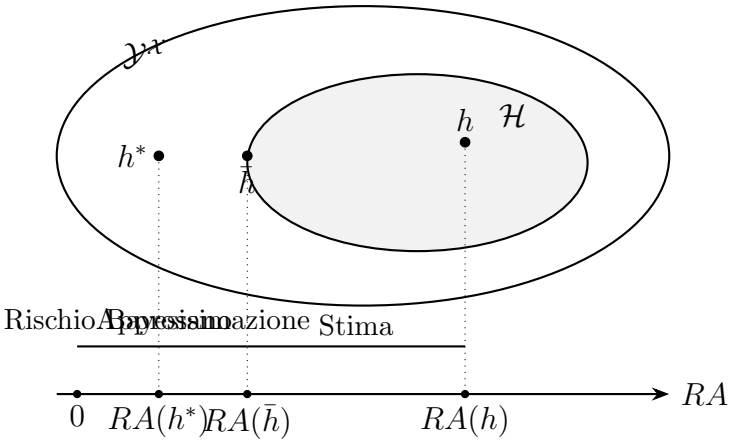


Figura 3: Rappresentazione geometrica dello spazio delle funzioni e decomposizione del Rischio Atteso in Rischio Bayesiano, Errore di Approssimazione ed Errore di Stima.

9 Modelli di Base e Metriche di Valutazione

9.1 Ipotesi di Base e Rischio Empirico

Il **Rischio Empirico (RE)** misura l'errore commesso dal modello sui dati di addestramento.

- **RE0 (Modello Costante):** È il modello più semplice in assoluto (baseline). Non dipendendo dalla variabile di input x, assume la forma $h(x) = c$. Per minimizzare l'errore quadratico medio, la costante c ottimale corrisponde semplicemente alla **media aritmetica** dei valori target del campione di addestramento.
- **RE1 (Modello Lineare):** Rappresenta la migliore ipotesi lineare della forma $h(x) = w_0 + w_1 x$. Cerca la retta che minimizza l'errore sui dati.

9.2 Il Coefficiente di Determinazione (R^2)

Il coefficiente R^2 viene spesso preferito al Rischio Empirico puro perché è standardizzato. Nella regressione lineare semplice corrisponde al quadrato del coefficiente di correlazione di Pearson. Si calcola come:

$$R^2 = 1 - \frac{\text{Varianza non spiegata (Residui)}}{\text{Varianza totale dei dati}}$$

Essendo un valore compreso tra 0 e 1 (sui dati di addestramento), fornisce una scala di facile interpretazione: **più si avvicina a 1, migliore è l'adattamento del modello**. *Attenzione:* questo range $[0, 1]$ è garantito solo sul dataset usato per addestrare il modello. Se valutato su un **Test Set** (dati esterni, mai visti dal modello), il valore di R^2 può risultare negativo. Un $R^2 < 0$ indica che il modello si comporta peggio del modello RE0 (la media costante). Tuttavia questo coefficiente è molto specifico per la regressione lineare.

10 Ingegnerizzazione delle Feature (Feature Engineering)

10.1 Codifica delle Variabili Qualitative

I modelli matematici richiedono input numerici. Le variabili qualitative (categoriche) devono essere codificate (One-Hot Encoding).

- **Binarie (es. Maschio/Femmina):** Basta una sola variabile che assume valore 0 o 1.
- **Categorie Multiple (es. 3 opzioni):** Assegnare valori come 0, 1, 2 introduce un errore concettuale, poiché il modello interpreterebbe un ordine e una proporzione inesistenti (es. l'opzione 2 verrebbe considerata "il doppio" dell'opzione 1). **Soluzione:** Si utilizzano k variabili binarie (o $k-1$ per evitare multicollinearità), dove ogni variabile rappresenta la presenza (1) o assenza (0) di una specifica categoria.

10.2 Effetti Congiunti e Modelli Non Lineari

Spesso le variabili interagiscono tra loro. Moltiplicando due variabili di input (es. $x_1 \times x_2$), si crea una nuova variabile che permette al modello lineare di catturare un **effetto congiunto** (interazione) che altrimenti verrebbe ignorato.

Per andare oltre la semplice regressione lineare e catturare andamenti complessi, si utilizza l'espansione in base:

$$\mathbf{x} \xrightarrow{\phi} \phi(\mathbf{x}) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^n \end{bmatrix}$$

L'ipotesi diventa quindi un prodotto riga per colonna $h(x) = \mathbf{w}^T \phi(\mathbf{x})$. In questo modo, il modello è **lineare nei parametri w** (quindi addestrabile con i classici e veloci algoritmi di regressione lineare), ma **non lineare rispetto agli input originali**. In questo caso la funzione ϕ può restituire un output vettoriale diverso dalle varie potenze di x, e può avere diverse forme a seconda dei casi.

11 Scelta della Funzione di Costo e MLE

11.1 Maximum Likelihood Estimation (MLE) e Derivazione OLS

Come scegliamo i pesi $\mathbf{w}$ (o θ) ottimali? Lo facciamo massimizzando la **Verosimiglianza** (Likelihood), ovvero la probabilità di osservare i nostri dati di output dati gli input.

Assumiamo che il legame tra le variabili di input x_i e l'output y_i sia descritto dalla funzione ipotesi $h_\theta(x_i)$ a cui si aggiunge un termine di errore ϵ_i:

$$y_i = h_\theta(x_i) + \epsilon_i$$

L'assunzione fondamentale della regressione ai minimi quadrati è che gli errori ϵ_i siano indipendenti e identicamente distribuiti (i.i.d.) secondo una distribuzione Normale (Gaussiana) con media nulla e varianza σ^2 costante, ovvero $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. La densità di probabilità dell'errore è:

$$p(\epsilon_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\epsilon_i^2}{2\sigma^2}\right)$$

Poiché $\epsilon_i = y_i - h_\theta(x_i)$, possiamo scrivere la probabilità di osservare il valore y_i dato x_i e parametrizzato da θ come:

$$p(y_i \mid x_i; \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - h_\theta(x_i))^2}{2\sigma^2}\right)$$

La funzione di Verosimiglianza (Likelihood) $L(\theta)$ è la probabilità congiunta di osservare tutti gli n dati. Essendo i dati i.i.d., è il prodotto delle singole probabilità:

$$L(\theta) = \prod_{i=1}^{n} p(y_i \mid x_i; \theta) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - h_\theta(x_i))^2}{2\sigma^2}\right)$$

Massimizzare $L(\theta)$ equivale a massimizzare la sua log-verosimiglianza $\ell(\theta) = \log L(\theta)$, che trasforma i prodotti in somme:

$$\ell(\theta) = \log\left(\prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - h_\theta(x_i))^2}{2\sigma^2}\right)\right)$$

$$\ell(\theta) = n \log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2$$

Il primo termine è una costante indipendente da θ. Pertanto, massimizzare $\ell(\theta)$ dipende solo dal secondo termine:

$$\max_\theta \left(-\frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2\right)$$

Massimizzare una quantità negativa equivale a minimizzarne il valore assoluto. Otteniamo quindi la minimizzazione della Somma degli Scarti Quadratici:

$$\min_\theta \sum_{i=1}^{n} (y_i - h_\theta(x_i))^2$$

Nota bene sul Rumore: Questa dimostrazione evidenzia un concetto fondamentale nel Machine Learning: *la funzione di costo non è una scelta assoluta, ma può (e deve) essere scelta in base ad uno studio probabilistico sulla distribuzione del rumore (gli errori) nei dati.* Se assumiamo che il rumore segua una distribuzione Gaussiana, la scelta ottima per massimizzare la verosimiglianza è l'errore quadratico (OLS). Se l'analisi del rumore suggerisse una distribuzione diversa (ad esempio, una distribuzione di Laplace, tipica in presenza di molti outlier), la massimizzazione della verosimiglianza condurrebbe matematicamente ad un'altra funzione di costo (come lo scarto assoluto LAD).

11.2 LAD (Scarto Assoluto) vs OLS (Scarto Quadratico)

- **Scarto Quadratico (L2 / OLS):** Facile da trattare numericamente e derivabile. Tuttavia, penalizza enormemente gli *outlier* (valori anomali) perché eleva al quadrato l'errore.
- **Scarto Assoluto (L1 / LAD):** Pesa gli errori linearmente. Un outlier ha ancora un peso, ma nettamente inferiore rispetto a L2, rendendo, in questi casi, il modello molto più **robusto**.

12 Modelli Non Parametrici: K-Nearest Neighbors (KNN)

I modelli non parametrici non hanno una forma predefinita determinata da un numero fisso di parametri (come i pesi **w** nella regressione). Memorizzano invece il set di dati di addestramento e derivano la predizione per prossimità.

Algoritmo KNN: Dato un nuovo punto di input, l'algoritmo calcola la distanza tra questo e tutti i punti nel dataset di addestramento. Seleziona i K punti più vicini e:

- **Regressione:** Calcola la media (o media pesata) dei valori target dei K vicini.
- **Classificazione:** Assegna la classe di maggioranza tra i K vicini.

Vantaggi e Svantaggi del KNN:

- **Flessibilità (Basso Bias):** Può modellare funzioni molto complesse.
- **Alta Varianza:** Modelli troppo flessibili (es. $K = 1$) sono molto sensibili alle fluttuazioni del singolo esempio di rumore. Aumentare K diminuisce la varianza a scapito del bias.
- **Costo Computazionale:** Veloce su dataset piccoli, ma molto lento in fase di predizione su dataset grandi (richiede di calcolare le distanze con tutti i punti salvati in memoria).
- La scelta della funzione di **distanza** (es. Euclidea) è cruciale e soffre l'aumento delle dimensioni (Curse of Dimensionality). A differenza del KNN, la regressione lineare è molto più robusta all'aumentare delle variabili.

13 Argomenti Avanzati

13.1 Pesatura degli Esempi (Instance Weighting)

In alcuni dataset, certi esempi possono ripetersi esattamente o avere un'importanza maggiore (es. misurazioni più affidabili). Invece di duplicare i record in memoria, si può assegnare un **peso** a ciascun esempio unico, modificando la funzione di costo affinché penalizzi maggiormente gli errori commessi sugli esempi con peso più alto. Supponiamo che l'esempio $(x^{(i)}, y^{(i)})$ compaia β_i volte.

Il rischio empirico (in questo caso, l'MSE) diventa:

$$\mathrm{RE}_S(h) = \frac{1}{\beta_1 + \cdots + \beta_m} \sum_{i=1}^{m} \beta_i (h(x^{(i)}) - y^{(i)})^2$$

13.2 Regressione Multi-Output

Quando si vogliono predire contemporaneamente più variabili di output (vettori invece di scalari), il vettore dei pesi **w** si generalizza in una **matrice dei pesi** W. Tuttavia, nella pratica e nelle librerie software più comuni, questo viene spesso implementato eseguendo semplicemente N regressioni lineari separate, una per ogni variabile di output, a meno che non si vogliano modellare esplicitamente le correlazioni tra gli output stessi.

14 Il Ruolo dell'Ottimizzazione nel Machine Learning

Nel Machine Learning (ML), la fase di addestramento (*training*) di un modello equivale quasi sempre alla risoluzione di un **problema di ottimizzazione matematica**. L'obiettivo fondamentale consiste nel minimizzare una opportuna **funzione di costo** (o funzione di perdita / rischio empirico) $g(w)$, definita sullo spazio dei parametri del modello $w \in \mathbb{R}^N$.

14.1 Formulazione Matematica

Un problema di minimizzazione globale si definisce formalmente come segue:

Definizione 14.1 (Problema di Minimizzazione Globale)**.** Dati uno spazio dei parametri $\mathbb{R}^N$ e una funzione obiettivo $g : \mathbb{R}^N \to \mathbb{R}$, si cerca un vettore di parametri $w^* \in \mathbb{R}^N$ tale che:

$$g(w^*) \leq g(w) \quad \forall w \in \mathbb{R}^N$$

Il punto w^* prende il nome di **minimo globale** della funzione g.

Esempio 14.1 (Regressione Lineare)**.** Nella regressione lineare semplice con una sola variabile di ingresso, la funzione ipotizzata è una retta $\hat{y} = w_0 + w_1 x$, dove w_0 rappresenta l'intercetta (*intercept*) e w_1 la pendenza (*slope*). Il vettore dei parametri è $w = (w_0, w_1)^\top \in \mathbb{R}^2$. La funzione di costo misura la discrepanza tra le predizioni $\hat{y}$ e i valori reali y sul dataset.

14.2 Minimi Globali vs. Minimi Locali

Dal punto di vista puramente teorico, si desidera individuare un minimo globale w^*. Tuttavia, nell'ottimizzazione applicata al Machine Learning, gli algoritmi numerici spesso non sono in grado di garantire il raggiungimento del minimo globale:

- **Minimo Locale:** Un punto w^* è un minimo locale se $g(w^*) \leq g(w)$ per tutti i punti w appartenenti a un opportuno intorno di w^*.
- **Modelli Complessi e Reti Neurali:** Nelle reti neurali profonde (*Deep Learning*), le funzioni di costo presentano superfici altamente non convesse, caratterizzate da un numero elevatissimo di minimi locali, punti di sella (*saddle points*) e altopiani (*plateau*). In questi contesti è computazionalmente impossibile garantire la convergenza al minimo globale; ci si accontenta di individuare un buon minimo locale che fornisca un basso errore di generalizzazione.

15 Metodi di Ricerca di Ordine Zero (Black-Box)

I metodi di ordine zero valutano esclusivamente il valore della funzione di costo $g(w)$ nei punti campionati, senza richiedere la conoscenza delle sue derivate (da cui il termine *Black-Box*).

15.1 Grid Search e Randomized Search

Due approcci elementari per l'esplorazione dello spazio dei parametri sono:

1. **Grid Search (Ricerca su Griglia):** Discretizza lo spazio dei parametri definendo una griglia regolare e valuta la funzione di costo $g(w)$ in ciascun nodo della griglia.

2. **Randomized Search (Ricerca Casuale):** Campiona casualmente $N_{samples}$ punti dallo spazio dei parametri secondo una distribuzione uniforme e individua il punto con il valore di costo minimo.

15.1.1 La Maledizione della Multidimensionalità (*Curse of Dimensionality*)

I metodi su griglia soffrono in modo drammatico dell'aumento della dimensione N dello spazio dei parametri. Se si suddivide ciascuna dimensione in k intervalli, il numero totale di valutazioni della funzione di costo cresce esponenzialmente come k^N:

- Per $N = 1$ e $k = 10 \implies 10$ valutazioni.
- Per $N = 3$ e $k = 10 \implies 10^3 = 1000$ valutazioni.
- Per modelli di Modern ML / Deep Learning con $N \approx 10^9$ (miliardi di parametri), la Grid Search richiede 10^{10^9} valutazioni, un numero infinitamente superiore agli atomi dell'universo osservabile.

Nonostante questo limite invalicabile per il training dei pesi principali, Grid Search e Randomized Search mantengono un ruolo utile nell'ottimizzazione degli **iperparametri** (dove il numero di variabili è limitato, ad esempio $N \leq 5$).

15.2 Randomized Local Search (RLS)

Per superare la rigidità della griglia globale, si utilizzano algoritmi basati sul **miglioramento iterativo** tramite ricerca locale. Si parte da un punto iniziale $w^{(1)}$ e si genera una sequenza di vettori $w^{(1)}, w^{(2)}, \ldots, w^{(t)}$ aggiornando la posizione al tempo t tramite un vettore spostamento $d^{(t)}$:

$$w^{(t+1)} = w^{(t)} + \eta d^{(t)}$$

dove $d^{(t)}$ rappresenta la direzione del passo e $\eta > 0$ è il **tasso di apprendimento** (*learning rate*), che ne scala la lunghezza.

Definizione 15.1 (Algoritmo Randomized Local Search)**.**

1. **Input:** Funzione $g(w)$, punto iniziale $w^{(1)}$, passo $\eta > 0$, direzioni casuali K.
2. **Inizializzazione:** Poniamo $t = 1$.
3. **Generazione Direzioni:** Genera K vettori direzione casuali $\{d^{(1)}, d^{(2)}, \ldots, d^{(K)}\}$.
4. **Valutazione Locale:** Calcola $g(w^{(t)} + \eta d^{(k)})$ per ciascun $k = 1, \ldots, K$.
5. **Aggiornamento:** Individua l'indice k^* che minimizza il costo. Se $g(w^{(t)} + \eta d^{(k^*)}) < g(w^{(t)})$, imposta:
$$w^{(t+1)} = w^{(t)} + \eta d^{(k^*)}$$
altrimenti mantieni $w^{(t+1)} = w^{(t)}$.
6. **Iterazione:** Incrementa t e ripeti fino a convergenza.

Osservazione 15.1 (Geometria delle Direzioni in RLS)**.** È importante notare che il numero K di direzioni casuali generate ad ogni passo non deve necessariamente coincidere con la dimensione N dello spazio. Ad esempio, in un piano bidimensionale ($N = 2$), è possibile testare $K = 3$ direzioni disposte a 120° l'una dall'altra. Tuttavia, all'aumentare delle dimensioni N, la probabilità che una direzione generata casualmente sia una direzione di reale discesa crolla esponenzialmente. Pertanto, i metodi di ordine 0 rimangono intrinsecamente limitati e inefficienti per l'ottimizzazione di modelli complessi.

16 Classificazione dei Metodi e Sviluppi di Taylor

I metodi di ottimizzazione numerica si classificano in base all'**ordine massimo delle derivate** di $g(w)$ impiegate per la scelta del passo.

16.1 Approssimazioni di Taylor (Caso Univariato $N = 1$)

Sia $g : \mathbb{R} \to \mathbb{R}$ una funzione sufficientemente regolare. Lo sviluppo di Taylor attorno a un punto $v \in \mathbb{R}$ fornisce le seguenti approssimazioni della funzione nell'intorno del punto:

- **Approssimazione di Ordine 1 (Retta Tangente):**

$$a_1(w) = g(v) + g'(v)(w - v)$$

- **Approssimazione di Ordine 2 (Parabola Tangente):**

$$a_2(w) = g(v) + g'(v)(w - v) + \frac{1}{2}g''(v)(w - v)^2$$

16.2 Approssimazioni di Taylor (Caso Multivariato $N \geq 1$)

In spazi multidimensionali $\mathbb{R}^N$, le derivate prime e seconde sono rappresentate rispettivamente dal vettore Gradiente e dalla matrice Hessiana.

Definizione 16.1 (Gradiente ed Hessiana)**.** Sia $g : \mathbb{R}^N \to \mathbb{R}$ due volte differenziabile.

- Il **Gradiente** di g nel punto $v \in \mathbb{R}^N$ è il vettore colonna $N \times 1$ delle derivate parziali prime:

$$\nabla g(v) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(v) \\ \frac{\partial g}{\partial w_2}(v) \\ \vdots \\ \frac{\partial g}{\partial w_N}(v) \end{bmatrix} \in \mathbb{R}^N$$

- La **Matrice Hessiana** di g nel punto $v \in \mathbb{R}^N$ è la matrice simmetrica $N \times N$ delle derivate parziali seconde:

$$\nabla^2 g(v) = \begin{bmatrix} \frac{\partial^2 g}{\partial w_1^2}(v) & \frac{\partial^2 g}{\partial w_1 \partial w_2}(v) & \cdots & \frac{\partial^2 g}{\partial w_1 \partial w_N}(v) \\ \frac{\partial^2 g}{\partial w_2 \partial w_1}(v) & \frac{\partial^2 g}{\partial w_2^2}(v) & \cdots & \frac{\partial^2 g}{\partial w_2 \partial w_N}(v) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 g}{\partial w_N \partial w_1}(v) & \frac{\partial^2 g}{\partial w_N \partial w_2}(v) & \cdots & \frac{\partial^2 g}{\partial w_N^2}(v) \end{bmatrix} \in \mathbb{R}^{N \times N}$$

Le approssimazioni locali di Taylor in $\mathbb{R}^N$ assumono la forma:

- **Approssimazione di Ordine 1 (Iperpiano Tangente):**

$$a_1(w) = g(v) + \nabla g(v)^\top (w - v)$$

- **Approssimazione di Ordine 2 (Quadrilatero / Paraboloide Tangente):**

$$a_2(w) = g(v) + \nabla g(v)^\top (w - v) + \frac{1}{2}(w - v)^\top \nabla^2 g(v)(w - v)$$

17 Condizioni di Ottimalità e Teoria della Convessità

17.1 Condizioni Necessarie del Primo Ordine

Se la funzione $g : \mathbb{R}^N \to \mathbb{R}$ è differenziabile, la condizione:

$$\nabla g(v) = 0$$

è una **condizione necessaria** affinché v sia un minimo globale (o locale) interno. Tuttavia, questa condizione **non è sufficiente**: identifica unicamente i **punti critici** (o stazionari) della funzione, che possono includere:

1. Minimi locali.
2. Massimi locali.
3. Punti di sella (*saddle points*).

17.2 Definizione e Caratterizzazioni delle Funzioni Convesse

La convessità è la proprietà fondamentale che trasforma l'annullamento del gradiente da condizione necessaria a condizione **necessaria e sufficiente** per un minimo globale.

Definizione 17.1 (Convessità di Ordine 0 – Geometrica)**.** Una funzione $f : \mathbb{R}^N \to \mathbb{R}$ è convessa se, per ogni $a, b \in \mathbb{R}^N$ e per ogni $\lambda \in [0, 1]$, soddisfa:

$$f(\lambda a + (1 - \lambda)b) \leq \lambda f(a) + (1 - \lambda) f(b)$$

Geometricamente, il segmento che congiunge due punti qualsiasi del grafico della funzione (*corda*) si trova sempre al di sopra del grafico stesso.

Definizione 17.2 (Convessità di Ordine 1 – Differenziabile)**.** Se $f : \mathbb{R}^N \to \mathbb{R}$ è differenziabile, f è convessa se e solo se per ogni $a, b \in \mathbb{R}^N$:

$$f(b) \geq f(a) + \nabla f(a)^\top (b - a)$$

Geometricamente, la funzione giace interamente al di sopra di qualsiasi suo iperpiano tangente.

Definizione 17.3 (Convessità di Ordine 2 – Due volte differenziabile)**.** Se $f : \mathbb{R}^N \to \mathbb{R}$ è due volte differenziabile, f è convessa se e solo se la sua matrice Hessiana $\nabla^2 f(x)$ è **semidefinita positiva** per ogni $x \in \mathbb{R}^N$, ossia tutti i suoi autovalori sono non-negativi ($\lambda_i \geq 0$).

17.3 Criteri Operativi di Convessità e Dimostrazione per le Norme

In pratica, la convessità di funzioni complesse si verifica combinando regole fondamentali:

- Ogni funzione affine / lineare $f(x) = a^\top x + b$ è convessa.
- Se f, g sono convesse e $c \geq 0$, allora $c \cdot f(x)$, $f(x) + g(x)$ e $\max(f(x), g(x))$ sono convesse.
- Se f è convessa e $h(x) = Ax + b$ è affine, la composizione $f(Ax + b)$ è convessa.
- Una forma quadratica $g(w) = \frac{1}{2} w^\top Q w + r^\top w + b$ (con Q simmetrica) è convessa se e solo se Q ha autovalori tutti ≥ 0.

Teorema 17.1 (Convessità delle Norme)**.** *Ogni norma $\|\cdot\| : \mathbb{R}^N \to \mathbb{R}$ è una funzione convessa.*

Dimostrazione. Siano $a, b \in \mathbb{R}^N$ e $\lambda \in [0, 1]$. Applicando la disuguaglianza triangolare e la proprietà di omogeneità della norma:

$$\|\lambda a + (1 - \lambda)b\| \leq \|\lambda a\| + \|(1 - \lambda)b\| = |\lambda| \|a\| + |1 - \lambda| \|b\|$$

Poiché $\lambda \in [0, 1]$, si ha $|\lambda| = \lambda$ e $|1 - \lambda| = 1 - \lambda$. Dunque:

$$\|\lambda a + (1 - \lambda)b\| \leq \lambda \|a\| + (1 - \lambda)\|b\|$$

La disuguaglianza di convessità è soddisfatta per ogni $a, b \in \mathbb{R}^N$. □

17.4 Importanza della Convessità nelle Funzioni di Costo

Teorema 17.2 (Condizione Necessaria e Sufficiente di Ottimalità)**.** *Se $g : \mathbb{R}^N \to \mathbb{R}$ è convessa e differenziabile, la condizione $\nabla g(w^*) = 0$ è necessaria e sufficiente affinché w^* sia un minimo globale. Inoltre, ogni minimo locale di una funzione convessa è anche un minimo globale.*

Dimostrazione. Sia w^* un punto stazionario, cioè $\nabla g(w^*) = 0$. Per la caratterizzazione del primo ordine di una funzione convessa, per qualsiasi punto $w \in \mathbb{R}^N$ vale:

$$g(w) \geq g(w^*) + \nabla g(w^*)^\top (w - w^*) = g(w^*) + 0^\top (w - w^*) = g(w^*)$$

Dunque $g(w^*) \leq g(w)$ per ogni $w \in \mathbb{R}^N$, dimostrando che w^* è un minimo globale. □

Nelle applicazioni pratiche, se la funzione di costo originale non è convessa (ad esempio la costo $0-1$ nella classificazione), la si sostituisce con una funzione di costo surrogata convessa (es. *Cross-Entropy Loss* o *Hinge Loss*) in modo da poter garantire l'ottimalità della discesa.

18 Algoritmo di Discesa del Gradiente (Gradient Descent)

L'algoritmo di Discesa del Gradiente (*Gradient Descent*, GD – Cauchy, 1847) è il metodo di ottimizzazione del primo ordine fondamentale nel Machine Learning.

18.1 Formulazione dell'Algoritmo

L'algoritmo esplora lo spazio spostandosi ad ogni passo nella direzione opposta al gradiente corrente, ossia lungo la direzione di massima decrescita locale.

Definizione 18.1 (Algoritmo Gradient Descent)**.**

1. **Input:** Funzione differenziabile $g(w)$, punto iniziale $w^{(1)}$, tasso di apprendimento $\eta > 0$, numero di passi T.
2. **Ciclo di Aggiornamento:** Per $t = 1, 2, \ldots, T$:
$$w^{(t+1)} = w^{(t)} - \eta \nabla g(w^{(t)})$$
3. **Selezione dell'Ottimo:** Restituisci il punto con il minimo valore di costo osservato:
$$w^{GD} = \arg \min_{w \in \{w^{(1)}, \ldots, w^{(T)}\}} g(w)$$

Osservazione 18.1 (Nota sulla Selezione del Risultato)**.** L'algoritmo salva storicamente i valori $g(w^{(t)})$ e restituisce il candidato migliore osservato anziché restituire automaticamente l'ultimo vettore $w^{(T+1)}$. Questa precauzione garantisce la robustezza dell'algoritmo nel caso in cui le ultime iterazioni soffrano di oscillazioni per via di un η non perfetto.

18.2 Il Tasso di Apprendimento (η)

Il parametro $\eta > 0$ determina la lunghezza del passo lungo la direzione $-\nabla g(w^{(t)})$:

- η **troppo piccolo:** L'algoritmo fa progressi estremamente lenti, richiedendo un numero elevatissimo di iterazioni per avvicinarsi al minimo.
- η **bilanciato:** L'algoritmo scende in modo fluido ed efficiente verso il minimo.
- η **troppo grande:** L'algoritmo "scavalca" il minimo (*overshooting*), provocando oscillazioni divergenti e instabilità numerica.

19 Teoremi e Dimostrazioni Rigorose di Convergenza del GD

In questa sezione riportiamo e analizziamo nel dettaglio la dimostrazione formale del teorema di convergenza per l'algoritmo di Gradient Descent.

19.1 Dimostrazione per Funzioni Convesse e Gradienti Limitati

Teorema 19.1 (Teorema di Convergenza di GD – Versione 1)**.** *Sia $g : \mathbb{R}^N \to \mathbb{R}$ una funzione convessa e differenziabile. Supponiamo che esistano due costanti $D, G > 0$ tali che:*

1. ***Distanza iniziale limitata:*** *$\|w^{(1)} - w^*\| \leq D$, dove w^* è un minimo globale.*
2. ***Gradiente limitato:*** *$\|\nabla g(w^{(t)})\| \leq G$ per ogni $t = 1, \ldots, T$.*

Scegliendo un tasso di apprendimento fisso pari a:

$$\eta = \frac{D}{G\sqrt{T}}$$

l'algoritmo Gradient Descent soddisfa la seguente stima sull'errore di convergenza:

$$g(w^{GD}) - g(w^*) \leq \frac{1}{T}\sum_{t=1}^{T}\left(g(w^{(t)}) - g(w^*)\right) \leq \frac{DG}{\sqrt{T}}$$

Dimostrazione. La dimostrazione sfrutta tre elementi fondamentali:

1. **Regola di aggiornamento dell'algoritmo GD**:

$$w^{(t+1)} = w^{(t)} - \eta\nabla g(w^{(t)}) \implies \nabla g(w^{(t)}) = \frac{1}{\eta}\left(w^{(t)} - w^{(t+1)}\right)$$

2. **Proprietà di convessità del primo ordine**: Poiché g è convessa, per ogni $w, z \in \mathbb{R}^N$ vale:

$$g(w) - g(z) \leq \nabla g(w)^\top (w - z)$$

Sostituendo $w = w^{(t)}$ e $z = w^*$, otteniamo:

$$g(w^{(t)}) - g(w^*) \leq \nabla g(w^{(t)})^\top \left(w^{(t)} - w^*\right)$$

3. **Identità algebrica vettoriale**: Per ogni coppia di vettori $a, b \in \mathbb{R}^N$, vale l'identità:

$$2a^\top b = \|a\|^2 + \|b\|^2 - \|a - b\|^2$$

Verifica dell'identità: Si noti che $\|a-b\|^2 = (a-b)^\top(a-b) = \|a\|^2 + \|b\|^2 - 2a^\top b$. Riorganizzando i termini si ottiene direttamente l'identità espressa.

Applichiamo l'identità geometrica ponendo $a = w^{(t)} - w^{(t+1)} = \eta\nabla g(w^{(t)})$ e $b = w^{(t)} - w^*$. Notiamo che:

$$a - b = \left(w^{(t)} - w^{(t+1)}\right) - \left(w^{(t)} - w^*\right) = w^* - w^{(t+1)}$$

Dunque:

$$2\left(w^{(t)} - w^{(t+1)}\right)^\top \left(w^{(t)} - w^*\right) = \left\|w^{(t)} - w^{(t+1)}\right\|^2 + \left\|w^{(t)} - w^*\right\|^2 - \left\|w^{(t+1)} - w^*\right\|^2$$

Sostituendo questa relazione nella disuguaglianza derivante dalla convessità:

$$\begin{aligned} g(w^{(t)}) - g(w^*) &\leq \nabla g(w^{(t)})^\top \left(w^{(t)} - w^*\right) \\ &= \frac{1}{\eta}\left(w^{(t)} - w^{(t+1)}\right)^\top \left(w^{(t)} - w^*\right) \\ &= \frac{1}{2\eta}\left(\left\|w^{(t)} - w^{(t+1)}\right\|^2 + \left\|w^{(t)} - w^*\right\|^2 - \left\|w^{(t+1)} - w^*\right\|^2\right) \end{aligned}$$

Ricordando che $w^{(t)} - w^{(t+1)} = \eta\nabla g(w^{(t)})$, si ha $\left\|w^{(t)} - w^{(t+1)}\right\|^2 = \eta^2\left\|\nabla g(w^{(t)})\right\|^2$. Semplificando per η:

$$g(w^{(t)}) - g(w^*) \leq \frac{\eta}{2}\left\|\nabla g(w^{(t)})\right\|^2 + \frac{1}{2\eta}\left(\left\|w^{(t)} - w^*\right\|^2 - \left\|w^{(t+1)} - w^*\right\|^2\right)$$

Sfruttando l'ipotesi di limitatezza del gradiente ($\left\|\nabla g(w^{(t)})\right\| \leq G$):

$$g(w^{(t)}) - g(w^*) \leq \frac{\eta G^2}{2} + \frac{1}{2\eta}\left(\left\|w^{(t)} - w^*\right\|^2 - \left\|w^{(t+1)} - w^*\right\|^2\right)$$

Sommiamo ora ambo i membri per tutte le iterazioni $t = 1, 2, \ldots, T$:

$$\sum_{t=1}^{T}\left(g(w^{(t)}) - g(w^*)\right) \leq \sum_{t=1}^{T}\frac{\eta G^2}{2} + \frac{1}{2\eta}\sum_{t=1}^{T}\left(\left\|w^{(t)} - w^*\right\|^2 - \left\|w^{(t+1)} - w^*\right\|^2\right)$$

La seconda sommatoria è una **somma telescopica**, dove tutti i termini intermedi si cancellano a vicenda:

$$\sum_{t=1}^{T}\left(\left\|w^{(t)} - w^*\right\|^2 - \left\|w^{(t+1)} - w^*\right\|^2\right) = \left\|w^{(1)} - w^*\right\|^2 - \left\|w^{(T+1)} - w^*\right\|^2$$

Otteniamo quindi:

$$\sum_{t=1}^{T}\left(g(w^{(t)}) - g(w^*)\right) \leq \frac{T\eta G^2}{2} + \frac{1}{2\eta}\left(\left\|w^{(1)} - w^*\right\|^2 - \left\|w^{(T+1)} - w^*\right\|^2\right)$$

Poiché $-\left\|w^{(T+1)} - w^*\right\|^2 \leq 0$ e per ipotesi $\left\|w^{(1)} - w^*\right\| \leq D$, possiamo maggiorare:

$$\sum_{t=1}^{T}\left(g(w^{(t)}) - g(w^*)\right) \leq \frac{T\eta G^2}{2} + \frac{D^2}{2\eta}$$

Dividendo entrambi i membri per T, si ottiene la stima per l'errore medio:

$$\frac{1}{T}\sum_{t=1}^{T}\left(g(w^{(t)}) - g(w^*)\right) \leq \frac{\eta G^2}{2} + \frac{D^2}{2\eta T}$$

Sostituiamo ora il valore teorico ottimale del tasso di apprendimento $\eta = \frac{D}{G\sqrt{T}}$ (si ottiene ottimizzando su η:

$$\frac{\eta G^2}{2} + \frac{D^2}{2\eta T} = \frac{\left(\frac{D}{G\sqrt{T}}\right)G^2}{2} + \frac{D^2}{2T\left(\frac{D}{G\sqrt{T}}\right)} = \frac{DG}{2\sqrt{T}} + \frac{DG}{2\sqrt{T}} = \frac{DG}{\sqrt{T}}$$

Infine, per la definizione di $w^{GD} = \arg\min_t g(w^{(t)})$, il valore minimo di un insieme è sempre inferiore o uguale alla sua media aritmetica:

$$g(w^{GD}) - g(w^*) = \min_{t=1\ldots T}\left(g(w^{(t)}) - g(w^*)\right) \leq \frac{1}{T}\sum_{t=1}^{T}\left(g(w^{(t)}) - g(w^*)\right) \leq \frac{DG}{\sqrt{T}}$$

La convergenza asymptotica decresce come $\mathcal{O}(1/\sqrt{T})$. □

19.2 Convergenza per Funzioni β-smooth

Definizione 19.1 (Funzione β-smooth)**.** Una funzione convessa e differenziabile $g : \mathbb{R}^N \to \mathbb{R}$ è detta β-smooth ($\beta > 0$) se il suo gradiente è Lipschitziano con costante β, ovvero se per ogni $x, y \in \mathbb{R}^N$:

$$g(y) - g(x) - \nabla g(x)^\top (y - x) \leq \frac{\beta}{2}\|x - y\|^2$$

Questo garantisce che la componente non lineare della funzione non cresca più rapidamente di una parabola con curvatura β.

Teorema 19.2 (Teorema di Convergenza di GD – Versione 2 per Funzioni Smooth)**.** *Sia $g : \mathbb{R}^N \to \mathbb{R}$ convessa e β-smooth. Impostando un tasso di apprendimento fisso $\eta = 1/\beta$, l'algoritmo Gradient Descent garantisce un tasso di convergenza più rapido pari a:*

$$g(w^{GD}) - g(w^*) \leq \frac{2\beta\|w^{(1)} - w^*\|^2}{T - 1}$$

L'errore decresce asintoticamente come $\mathcal{O}(1/T)$.

Osservazione 19.1 (Riflessione Pratica sui Teoremi Teorici)**.** Sebbene i teoremi sopra riportati forniscano formule analitiche per η (es. $\eta = \frac{D}{G\sqrt{T}}$ o $\eta = 1/\beta$), nella pratica applicativa del Machine Learning le costanti D, G, β **non sono quasi mai note a priori**. Pertanto, la scelta del valore di η rimane uno dei principali **iperparametri** da sintonizzare empiricamente tramite tecniche di Cross-Validation.

20 Problematiche del GD e Varianti Avanzate

Nonostante le garanzie teoriche, l'algoritmo di Gradient Descent standard presenta due problematiche operative cruciali:

20.1 Problema 1: Movimento a Zig-Zag

Quando le curve di livello della funzione di costo sono allungate o mal condizionate (presenza di valli strette ed ellittiche), il gradiente $-\nabla g(w)$ è quasi ortogonale alla direzione ideale verso il minimo. Questo provoca un andamento oscillatorio a **zig-zag**, che rallenta drasticamente la convergenza.

20.2 Problema 2: Rallentamento nei Punti Critici e Plateau

Nelle regioni della superficie di costo molto piatte (dove $\|\nabla g(w)\| \approx 0$), la grandezza del passo $\eta\nabla g(w)$ si contrae fortemente. L'algoritmo inizia a muoversi a velocità quasi nulla, fermandosi precocemente lontano dal punto di ottimo.

20.3 Varianti del Primo Ordine

Per mitigare queste problematiche, le librerie moderne di ML utilizzano estensioni del GD:

- **Momentum:** Aggiunge all'aggiornamento corrente una quota del vettore di spostamento precedente:

$$v^{(t+1)} = \beta v^{(t)} + \eta \nabla g(w^{(t)}), \quad w^{(t+1)} = w^{(t)} - v^{(t+1)}$$

  Questo accumula velocità nelle direzioni consistenti e smorza le oscillazioni a zig-zag.

- **Gradiente Normalizzato:** Utilizza la direzione $-\frac{\nabla g(w)}{\|\nabla g(w)\|}$, garantendo passi di lunghezza costante anche su superfici piatte.

- **Adagrad / RMSprop / Adam:** Adattano dinamicamente il tasso di apprendimento η in maniera specifica per ciascun singolo parametro, basandosi sulla memoria storica dei gradienti passati.

21 Metodi del Secondo Ordine e Quasi-Newton

21.1 Il Metodo di Newton-Raphson

I metodi del secondo ordine utilizzano l'approssimazione quadratica locale di Taylor, sfruttando sia il Gradiente sia la Matrice Hessiana.

Definizione 21.1 (Metodo di Newton-Raphson)**.** Data una funzione due volte differenziabile $g(w)$, l'aggiornamento al passo t si ottiene azzerando il gradiente dell'approssimazione quadratica $a_2(w)$:

$$w^{(t+1)} = w^{(t)} - \left[\nabla^2 g(w^{(t)})\right]^{-1} \nabla g(w^{(t)})$$

21.2 Limiti dei Metodi del Secondo Ordine nel ML

Nonostante il metodo di Newton richieda pochissime iterazioni per convergere (convergenza quadratica), il suo impiego nel Machine Learning è fortemente limitato da tre fattori:

1. **Occupazione di Memoria $\mathcal{O}(N^2)$:** Per modelli con $N = 10^8$ parametri, memorizzare la matrice Hessiana $N \times N$ richiede 10^{16} numeri in virgola mobile ($\sim$ 80 Petabyte di RAM).

2. **Costo Computazionale dell'Inversione $\mathcal{O}(N^3)$:** Risolvere il sistema lineare o invertire una matrice $N \times N$ ad ogni singola iterazione ha un costo computazionale proibitivo.

3. **Requisito di Convessità:** L'Hessiana deve essere definita positiva per garantire che la direzione individuata sia di discesa. In contesti non convessi, l'Hessiana può avere autovalori negativi, spingendo l'algoritmo verso massimi locali o punti di sella.

21.3 Metodi Quasi-Newton (BFGS e L-BFGS)

I metodi **Quasi-Newton** evitano il calcolo esplicito dell'Hessiana mantenendo un'approssimazione dell'inversa dell'Hessiana $B_t \approx [\nabla^2 g(w)]^{-1}$, aggiornata iterativamente tramite la variazione dei gradienti.

- **BFGS (Broyden-Fletcher-Goldfarb-Shanno):** Costruisce e aggiorna iterativamente una matrice $N \times N$.

- **L-BFGS (Limited-Memory BFGS):** Non memorizza mai la matrice $N \times N$, ma ricostruisce l'azione dell'inversa dell'Hessiana attraverso uno storico limitato degli ultimi m vettori dei parametri e dei gradienti ($m \approx 10 \dots 20$), riducendo l'occupazione di memoria a $\mathcal{O}(mN)$.

Osservazione 21.1 (Classificazione di L-BFGS)**.** Sebbene i metodi Quasi-Newton nascano per simulare il comportamento del metodo del secondo ordine di Newton, **utilizzano esclusivamente i valori della funzione e del gradiente**. Pertanto, dal punto di vista formale, **BFGS e L-BFGS sono algoritmi del primo ordine**.

22 Ottimizzazione del Rischio Empirico e Varianti Stocastiche

22.1 Formulazione del Rischio Empirico nel ML

Nel Machine Learning supervisionato, con un dataset di training di m esempi $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ la funzione di costo globale $g(w)$ è la media delle perdite calcolate su ciascun esempio:

$$g(w) = \frac{1}{m} \sum_{i=1}^{m} \ell_i(w) = \frac{1}{m} \sum_{i=1}^{m} \ell(h_w(x^{(i)}), y^{(i)})$$

Il gradiente complessivo è la media dei gradienti individuali:

$$\nabla g(w) = \frac{1}{m} \sum_{i=1}^{m} \nabla \ell_i(w)$$

22.2 Batch Gradient Descent (GD)

L'algoritmo GD classico (detto **Batch GD**) calcola il gradiente esatto sommando i contributi di tutti gli m esempi del dataset ad ogni singolo passo:

$$w^{(t+1)} = w^{(t)} - \eta \cdot \frac{1}{m} \sum_{i=1}^{m} \nabla \ell_i(w^{(t)})$$

Quando m è molto grande (milioni di dati), il calcolo del gradiente "batch" diventa estremamente lento.

22.3 Stochastic Gradient Descent (SGD)

L'algoritmo **Stochastic Gradient Descent (SGD)** sostituisce la media globale con il gradiente di un singolo esempio i estratto casualmente dal dataset ad ogni iterazione.

Definizione 22.1 (Algoritmo SGD)**.** Per $t = 1, \dots, T$:

- Estrai un indice $i \in \{1, \dots, m\}$ in modo uniforme a caso.
- Aggiorna i pesi considerandone solo il gradiente sul dato i:

$$w^{(t+1)} = w^{(t)} - \eta \nabla \ell_i(w^{(t)})$$

22.3.1 Esempio 1: SGD per la Regressione Lineare (Regola LMS)

Consideriamo l'ipotesi lineare $\hat{y} = h(x) = w^\top x$ e la funzione di costo quadratico sull'esempio i:

$$\ell_i(w) = (\hat{y}^{(i)} - y^{(i)})^2 = (w^\top x^{(i)} - y^{(i)})^2$$

Calcoliamo il gradiente rispetto al vettore dei pesi w:

$$\nabla \ell_i(w) = 2(w^\top x^{(i)} - y^{(i)})x^{(i)} = 2(\hat{y}^{(i)} - y^{(i)})x^{(i)}$$

La regola di aggiornamento di SGD prende il nome di **Regola Least-Mean-Squares (LMS)**:

$$w \leftarrow w - 2\eta(\hat{y}^{(i)} - y^{(i)})x^{(i)}$$

22.3.2 Esempio 2: SGD per la Regressione LAD (Norma L_1)

Consideriamo l'ipotesi lineare $\hat{y} = w^\top x$ con funzione di costo **LAD** (*Least Absolute Deviations*):

$$\ell_i(w) = |\hat{y}^{(i)} - y^{(i)}| = |w^\top x^{(i)} - y^{(i)}|$$

Poiché la funzione valore assoluto non è differenziabile in zero, si calcola il **subgradiente**:

$$\partial \ell_i(w) = \text{sgn}(\hat{y}^{(i)} - y^{(i)})x^{(i)}$$

dove la funzione segno è definita elemento per elemento come $\text{sgn}(z) = +1$ se $z > 0$ e -1 se $z \leq 0$. La regola di aggiornamento diventa:

$$w \leftarrow w - \eta \cdot \text{sgn}(\hat{y}^{(i)} - y^{(i)})x^{(i)}$$

22.4 Mini-Batch SGD

Mini-Batch SGD rappresenta il compromesso ottimale tra Batch GD e SGD puro. Ad ogni iterazione, si estrae un piccolo sottoinsieme (lotto) di dati $B \subset \{1, \ldots, m\}$ di dimensione $|B|$ (tipicamente $|B| = 32, 64, 128, 256$):

$$w^{(t+1)} = w^{(t)} - \eta \cdot \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(w^{(t)})$$

- Se $|B| = 1 \implies$ si ottiene **SGD puro**.
- Se $|B| = m \implies$ si ottiene **Batch GD**.
- Per $1 < |B| < m \implies$ si riduce il rumore di stima rispetto a SGD e si sfrutta appieno il **parallelismo vettoriale hardware** delle moderne GPU.

22.5 Confronto Sintetico tra le Varianti

22.6 Note Implementative in `scikit-learn`

In Python con la libreria `scikit-learn`, l'interfaccia `SGDRegressor` implementa l'algoritmo di discesa stocastica del gradiente (con batch di dimensione $|B| = 1$):

- **Costo Quadratico Standard:** `SGDRegressor(loss='squared_error', eta0=0.01)`

Proprietà	Batch GD	SGD (Puro)	Mini-Batch SGD
Costo per passo	$\mathcal{O}(m \cdot N)$ (Elevato)	$\mathcal{O}(N)$ (Molto Basso)	$\mathcal{O}(\|B\| \cdot N)$ (Ottimizzato)
Traiettoria	Liscia e deterministica	Molto rumorosa / incerta	Rumorosità moderata
Parallelismo GPU	Non sempre efficiente	Assente	Eccellente
Localizzazione Ottimo	Molto precisa	Oscillante attorno all'ottimo	Buona ed efficiente

Tabella 1: Confronto tra le modalità di calcolo del gradiente.

- **Costo Robusto (Huber):** `SGDRegressor(loss='huber', eta0=0.01)`

Osservazione 22.1 (Gestione del Learning Rate in `scikit-learn`)**.** Di default, `scikit-learn` adotta un tasso di apprendimento variabile nel tempo secondo la legge `learning_rate='invscaling'`:

$$\eta(t) = \frac{\eta_0}{t^{1/4}}$$

Per mantenere un tasso di apprendimento costante pari a η_0, occorre impostare esplicitamente l'opzione `learning_rate='constant'`.

23 Partizionamento dei Dati e Selezione degli Iperparametri

23.1 Divisione in Test e Train Set

L'obiettivo centrale di un algoritmo di apprendimento automatico supervisionato consiste nell'individuare una funzione di predizione (ipotesi) $h \in \mathcal{H}$ capace di generalizzare correttamente su dati non osservati, minimizzando il rischio atteso (o errore di generalizzazione):

$$R(h) = \mathbb{E}_{(X,Y)\sim\mathcal{D}}[L(h(X), Y)] \tag{26}$$

dove $\mathcal{D}$ è la distribuzione di probabilità congiunta (e ignota) del processo generativo dei dati e $L(\cdot,\cdot)$ indica la funzione di perdita (*loss function*). Poiché $\mathcal{D}$ non è direttamente accessibile, come detto, si stima il rischio atteso valutando il rischio empirico su un set, chiamato test set T, che sia separato dal set usato per addestrare il modello (quello di train):

$$RE_T(h) = \frac{1}{m}\sum_{i=1}^{m} L(h(x_i), y_i) \tag{27}$$

In quest'ottica, dal dataset originale è necessario ricavare due set, ma nel farlo, per evitare di introdurre dipendenze tra i dati, gli esempi vanno separati casualmente. In Python, la funzione `train_test_split` del modulo `scikit-learn` effettua questa operazione tramite uno shuffling preventivo seguito da una suddivisione disgiunta.

23.2 Selezione degli Iperparametri e Grid Search

I modelli di apprendimento dipendono da due distinte classi di quantità:

a) Parametri del modello (w): appresi direttamente dai dati mediante l'algoritmo di ottimizzazione (es. i pesi di una regressione o di una rete neurale).

b) Iperparametri ($\theta \in \Theta$): configurati esternamente dall'utente prima dell'addestramento (es. il tasso di apprendimento η e il numero di iterazioni T nel Gradient Descent, il parametro K nel K-Nearest Neighbors, o il coefficiente di penalizzazione λ).

La dipendenza della bontà del modello dagli iperparametri non è quasi mai esprimibile in forma analitica differenziabile; pertanto, la loro ottimizzazione si avvale di metodi di ordine zero. La strategia esaustiva più comune è la Grid Search (`GridSearchCV`), che definisce una griglia discreta di combinazioni $\Theta = \Theta_1 \times \Theta_2 \times \cdots \times \Theta_k$ ed esegue il training del modello su ciascuna tupla.

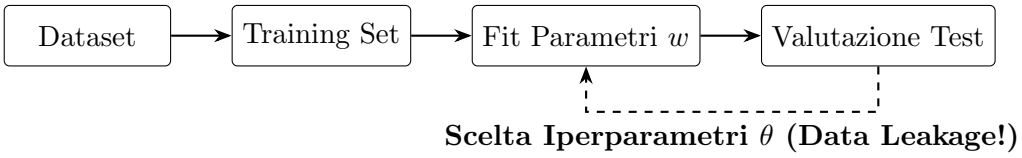


Figura 4: Contaminazione del Test Set (*Data Leakage*) causata dall'utilizzo della metrica di test per scegliere la combinazione ottimale degli iperparametri.

Osservazione 23.1 (Rischio di Data Leakage)**.** Qualora l'aggiustamento degli iperparametri venga condotto valutando le performance direttamente sul Test Set (Figura 4), l'informazione del test set si trasmette indirettamente al modello. Il test set perde la sua neutralità e il rischio empirico calcolato su di esso diventa uno stimatore distorto del reale rischio atteso.

23.3 Architettura a Tre Insiemi

Per evitare distorsioni nella stima finale, il dataset iniziale deve essere suddiviso in tre insiemi mutualmente disgiunti:

- Training Set: destinato esclusivamente all'ottimizzazione dei parametri interni del modello w.
- Validation Set: impiegato per confrontare le diverse configurazioni di iperparametri θ e selezionare l'architettura migliore.
- Test Set: mantenuto rigorosamente secretato fino al termine della fase di sviluppo e utilizzato una sola volta per misurare la capacità di generalizzazione non distorta dell'ipotesi finale.

23.4 Dimensionamento dei Set e Teoria della Stima

La dimensione del Validation e del Test Set deve essere calibrata in modo da garantire la precisione desiderata nella stima del rischio atteso, riservando la massima percentuale possibile di dati al Training Set.

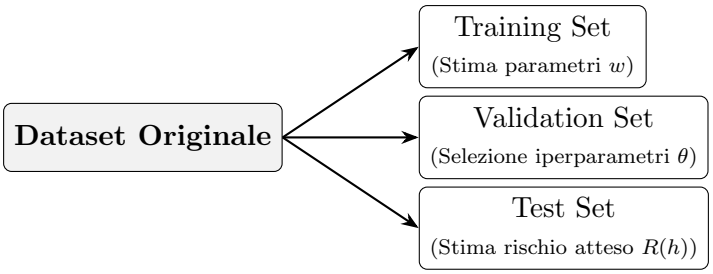


Figura 5: Partizionamento corretto del dataset in tre sottoinsiemi disgiunti.

Teorema 23.1 (Dimensionamento del Test Set - Disuguaglianza di Hoeffding)**.** *Sia h un'ipotesi fissata e la funzione di perdita assuma valori nell'intervallo $[0, C]$. Per ogni $\delta \in (0, 1)$, con probabilità almeno $1 - \delta$ rispetto al campionamento casuale di un Test Set T di cardinalità m_T, si verifica:*

$$|RE_T(h) - RA(h)| \leq C\sqrt{\frac{\ln(2/\delta)}{2m_T}} \tag{28}$$

L'equazione (28) attesta che l'errore di stima decresce secondo la legge $O(1/\sqrt{m_T})$:

- Per una tolleranza sull'errore pari a $\pm 0.1C$, è sufficiente una cardinalità $m_T \approx 100$ esempi.
- Per una precisione elevata pari a $\pm 0.01C$, occorrono $m_T \approx 10.000$ esempi.

23.4.1 Come si Valuta la Grandezza di Validation e Test Set

Sia $l_i = L(h(x_i), y_i)$ il valore della perdita misurato sul singolo esempio i-esimo del test set per $i = 1, \ldots, m_T$. Il rischio empirico sul test set $\hat{R}_{test}(h)$ è definito come la media campionaria delle perdite:

$$\hat{R}_{test}(h) = \bar{l} = \frac{1}{m_T}\sum_{i=1}^{m_T} l_i \tag{29}$$

Assumendo le perdite l_i come variabili aleatorie i.i.d. con varianza teorica della popolazione pari a σ_L^2, la varianza dello stimatore $\hat{R}_{test}(h)$ vale:

$$\mathrm{Var}(\hat{R}_{test}) = \mathrm{Var}\left(\frac{1}{m_T}\sum_{i=1}^{m_T} l_i\right) = \frac{1}{m_T^2}\sum_{i=1}^{m_T}\mathrm{Var}(l_i) = \frac{m_T\sigma_L^2}{m_T^2} = \frac{\sigma_L^2}{m_T} \tag{30}$$

Lo stimatore non distorto per la varianza della singola osservazione σ_L^2 è rappresentato dalla varianza campionaria s_L^2:

$$s_L^2 = \frac{1}{m_T - 1}\sum_{i=1}^{m_T}(l_i - \bar{l})^2 \tag{31}$$

Di conseguenza, lo stimatore della varianza della media campionaria (ovvero l'incertezza sulla stima del rischio atteso) è:

$$\widehat{\mathrm{Var}}(\hat{R}_{test}) = \frac{s_L^2}{m_T} \tag{32}$$

Quando si affronta un problema di questo tipo, si ha una certa soglia di precisione, perciò se le stime ottenute non rientrano in tale soglia, è necessario aumentare la grandezza dei nostri set di test e validation. Mentre il training set deve essere il più grande possibile.

23.5 Validazione Incrociata (K-Fold Cross-Validation)

Nelle situazioni in cui la scarsità dei dati impedisce la creazione di un Validation Set dedicato senza penalizzare gravemente l'addestramento, si ricorre alla K-Fold Cross-Validation.



Figura 6: Schema di funzionamento della K-Fold Cross-Validation per $K = 3$.

Definizione 23.1 (Procedura K-Fold Cross-Validation)**.** Dato un Training Set S partizionato in K sottoinsiemi disgiunti di cardinalità pressoché identica $S_1, S_2, \ldots, S_K$:

1. Per ogni $i = 1, \ldots, K$:
   (a) Addestra l'ipotesi h_i sull'insieme formato da tutti i dati tranne quelli del fold i-esimo ($S \setminus S_i$).
   (b) Misura il rischio empirico $RE_{S_i}(h_i)$ sul fold escluso S_i.
2. Calcola la media dei rischi di validazione sui K fold come stima finale del rischio atteso:

$$RA(h) \approx \frac{1}{K} \sum_{i=1}^{K} RE_{S_i}(h_i) \tag{33}$$

Questo metodo non ha della teoria che lo supporta eccetto in casi molto specifici, tuttavia sembre essere l'unica scappatoia quando c'è carenza di dati.

24 Metodi di Riduzione delle Feature

In presenza di un numero elevato di variabili di input d, i modelli soffrono dell'incremento della varianza e della drastica perdita di interpretabilità. Le principali metodologie di contenimento della dimensionalità si dividono in Selezione, Regolarizzazione e Proiezione.

24.1 Selezione di Sottoinsiemi (Feature Selection)

24.1.1 Best Subset Selection

L'algoritmo valuta in maniera esaustiva ogni possibile combinazione di variabili. Per ogni dimensione del sottoinsieme $k \in \{0, 1, \ldots, d\}$, si addestrano tutte le $\binom{d}{k}$ ipotesi, scegliendo la migliore rispetto al rischio empirico.

Osservazione 24.1 (Esplosione Combinatoria)**.** Il numero complessivo di modelli da addestrare tramite Best Subset Selection è dato da:

$$\sum_{k=0}^{d} \binom{d}{k} = 2^d \tag{34}$$

La complessità computazionale esponenziale $O(2^d)$ limita l'applicazione del metodo a valori di d piccoli, oltre a determinare un elevato rischio di overfitting sullo spazio delle sotto-ipotesi ipotizzabili.

24.1.2 Selezione Passo-Passo Sequenziale (Stepwise Selection)

Per abbattere la complessità a livello polinomiale $O(d^2)$, si utilizzano strategie greedy:

1. Forward Stepwise Selection: Inizia dal modello costante privo di variabili (h_0). A ogni passo k, valuta i $d-k$ modelli ottenibili integrando una singola variabile aggiuntiva e seleziona quella che massimizza la diminuzione dell'errore empirico. In questo caso si salva il rischio empirico ottenuto per ogni passo e si prende quello minore, scegliendo quindi il relativo numero di features. Naturalmente non vi è alcuna garanzia che il rischio così ottenuto sia il più basso rispetto alle 2^d possibilità, ma in caso di molte features è l'unica strada praticabile.

2. Backward Stepwise Selection: Parte dal modello completo contenente tutte le d variabili (h_d). Ad ogni iterazione, elimina la variabile la cui assenza causa il minore incremento dell'errore. Risulta preferibile alla selezione in avanti se $d - d' < d'$.

24.1.3 Selezione Basata sui Coefficienti

Per i modelli dotati di parametri associati direttamente agli input (es. Regressione Lineare), è possibile addestrare il modello completo, ordinare le feature in base all'ampiezza in valore assoluto dei coefficienti $|w_j|$, e conservare unicamente le k variabili con magnitudo dominante (`SelectFromModel` in `scikit-learn`).

25 Metodi di Regolarizzazione (Shrinkage)

I metodi di regolarizzazione modificano la funzione obiettivo introducendo un termine di penalità legato alla complessità del modello, bilanciando in modo automatico il trade-off bias-varianza.

Definizione 25.1 (Minimizzazione del Rischio Regolarizzato)**.** Sia $R(w) \geq 0$ una funzione di regolarizzazione definita sul vettore dei parametri. Il problema di ottimizzazione assume la forma:

$$\min_{w \in \mathbb{R}^{d+1}} \left(RE_S(w) + R(w) \right) \tag{35}$$

In pratica si tende a preferire un modello meno complesso anche se, in misura non esagerato, ritorna un rischio empirico maggiore.

25.1 Regressione Ridge (Regolarizzazione L_2)

La regressione Ridge adotta la norma L_2 al quadrato del vettore dei parametri come penalizzazione, escludendo il termine noto di intercetta w_0:

$$\min_{w \in \mathbb{R}^{d+1}} \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|w\|_2^2 \quad \text{con } \lambda \geq 0 \tag{36}$$

Proposizione 25.1 (Soluzione Analitica di Ridge)**.** *Ponendo pari a zero il gradiente della funzione di costo, si ottiene la soluzione analitica in forma chiusa:*

$$w^* = \left(X^T X + \lambda m I_0 \right)^{-1} X^T y \tag{37}$$

dove $I_0 = \text{diag}(0, 1, 1, \ldots, 1)$ *evita la regolarizzazione del bias* w_0*.*

25.1.1 Impatto del Parametro λ sui Coefficienti in Ridge

L'iperparametro λ agisce come un fattore di contrazione (*shrinkage*) continuo sull'ampiezza dei pesi w_j:

- Caso $\lambda = 0$: Il termine di penalizzazione si annulla. La soluzione coincide esattamente con lo stimatore dei minimi quadrati ordinari (OLS), $w^*_{\text{OLS}} = (X^T X)^{-1} X^T y$. Il modello esprime la massima capacità adattiva sui dati di training (minimo bias di addestramento, elevata varianza).
- Caso $\lambda \to \infty$: La penalità $\lambda \|w\|_2^2$ domina incondizionatamente il rischio empirico. Per evitare un costo infinito, tutti i coefficienti associati alle feature di input si annullano asintoticamente ($\lim_{\lambda \to \infty} w_j = 0$ per $j = 1, \ldots, d$). Il modello degenera nell'ipotesi banale $h(x) = w_0 = \bar{y}$ (massimo bias, varianza nulla).

25.2 Regressione LASSO (Regolarizzazione L_1)

La regressione LASSO (Least Absolute Shrinkage and Selection Operator) applica la norma L_1 al vettore dei parametri, sempre escludendo l'intercetta w_0:

$$\min_{w \in \mathbb{R}^{d+1}} \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|w\|_1 \quad \text{con } \|w\|_1 = \sum_{j=1}^{d} |w_j| \tag{38}$$

Teorema 25.1 (Convessità del Problema LASSO)**.** *La funzione di costo del LASSO $J(w) = \frac{1}{m}\|Xw - y\|_2^2 + \lambda \|w\|_1$ è strettamente convessa nell'intero dominio $\mathbb{R}^{d+1}$.*

Dimostrazione. Consideriamo le due componenti distinte:

1. $g(w) = \frac{1}{m}\|Xw - y\|_2^2$ è una forma quadratica con matrice Hessiana $\nabla^2 g(w) = \frac{2}{m} X^T X$. Poiché per ogni vettore non nullo $v \in \mathbb{R}^{d+1}$ si ha $v^T (X^T X) v = \|Xv\|_2^2 \geq 0$, la matrice Hessiana è semidefinita positiva, provando la convessità di $g(w)$.
2. $h(w) = \lambda \sum_{j=1}^{d} |w_j|$ è una combinazione lineare a coefficienti positivi ($\lambda \geq 0$) di funzioni valore assoluto, ciascuna delle quali soddisfa la disuguaglianza di convessità per la disuguaglianza triangolare: $|ta + (1-t)b| \leq t|a| + (1-t)|b|$ per $t \in [0, 1]$.

La somma di funzioni convesse è convessa. Di conseguenza $J(w)$ è convessa su $\mathbb{R}^{d+1}$, il che garantisce che qualsiasi minimo locale trovato da algoritmi di ottimizzazione (es. Gradient Descent o Coordinate Descent) sia globale. □

25.2.1 Impatto di λ nel LASSO

A differenza di Ridge, la penalizzazione L_1 esercita una contrazione non lineare dotata di selezione automatica delle variabili:

- Evoluzione con λ: All'aumentare di λ, le traiettorie dei coefficienti (*coefficient paths*) incontrano l'asse zero in corrispondenza di valori critici finiti di λ, rimuovendo progressivamente le feature meno rilevanti dal modello ed effettuando una vera e propria *Feature Selection*.

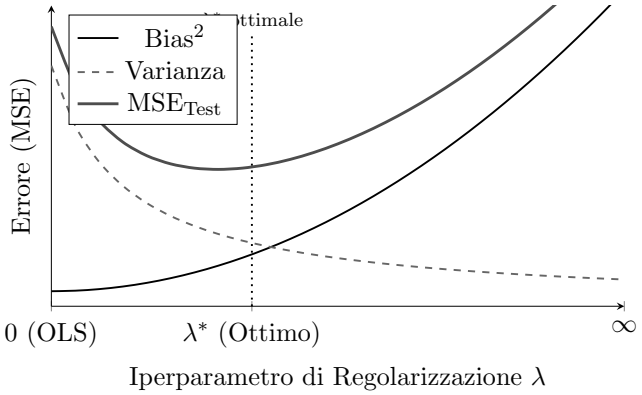


Figura 7: Andamento di Bias, Varianza ed Errore di Test al variare dell'iperparametro λ. A sinistra ($\lambda = 0$) domina la varianza per overfitting; a destra ($\lambda \to \infty$) domina il bias per underfitting.

25.3 Trade-Off Bias-Varianza ed Evoluzione dei Modelli

La scelta di λ governa in maniera diretta la posizione del modello lungo la curva del trade-off bias-varianza:

Come sintetizzato nella Figura 7:

1. Zona di Under-regolarizzazione ($\lambda \to 0$): I parametri w sono liberi di assumere valori elevati per minimizzare l'errore sul training set. La varianza dello stimatore è alta, rendendo il modello ipersensibile alle fluttuazioni casuali del campione (rischio di overfitting).

2. Zona Ottimale ($\lambda = \lambda^*$): Si individua tramite Validation Set o K-Fold Cross Validation la combinazione che minimizza il rischio atteso stimato. I parametri vengono contratti sufficientemente da abbattere la varianza senza introdurre un bias eccessivo.

3. Zona di Sovra-regolarizzazione ($\lambda \to \infty$): La penalità soffoca la capacità espressiva del modello. I pesi w tendono a zero, la varianza crolla ma il bias cresce quadraticamente (rischio di underfitting).

25.4 Standardizzazione delle Feature

A differenza della regressione dei minimi quadrati ordinari (OLS), dove i coefficienti risultano equivarianti per scala, la penalizzazione della norma in Ridge e LASSO risente dell'unità di misura delle variabili. Se una feature v possiede una scala molto grande, il corrispondente peso w_j sarà naturalmente piccolo per compensare la magnitudo dell'input, subendo una penalizzazione sbilanciata. Risulta quindi indispensabile effettuare la standardizzazione preventiva trasformando ogni feature v:

$$\tilde{v}_i = \frac{v_i - \bar{v}}{\sqrt{\frac{1}{m}\sum_{j=1}^{m}(v_j - \bar{v})^2}} \tag{39}$$

26 Sintesi delle Interfacce in Scikit-Learn

Tabella 2: Corrispondenza tra i concetti teorici e le classi della libreria `scikit-learn`.

Concetto Teorico	**Modulo / Classe Scikit-Learn**	**Principali Iperparametri**
Splitting Dati	`train_test_split`	`test_size`, `shuffle`
Grid Search	`GridSearchCV`	`param_grid`, `cv`, `scoring`
Cross-Validation	`KFold`	`n_splits`, `shuffle`
Feature Selection Passo-Passo	`SequentialFeatureSelection`	`n_features_to_select`, `directi`
Selezione da Modello	`SelectFromModel`	`max_features`, `threshold`
Regolarizzazione L_2 (Ridge)	`Ridge` / `SGDRegressor`	`alpha` ($= m\lambda$), `penalty='l2'`
Regolarizzazione L_1 (LASSO)	`Lasso` / `SGDRegressor`	`alpha` ($= \lambda/2$), `penalty='l1'`

27 Introduzione ai Problemi di Classificazione

Nel contesto dell'apprendimento supervisionato, i problemi di classificazione mirano a dedurre una regola di predizione (o ipotesi) $h : \mathcal{X} \to \mathcal{Y}$ in grado di mappare uno spazio degli input $\mathcal{X}$ (es. l'insieme dei pixel di un'immagine) in uno spazio degli output qualitativo e discreto $\mathcal{Y}$ (es. le etichette delle cifre da 0 a 9).

27.1 Funzione di Costo e Valutazione del Rischio

Per quantificare matematicamente le prestazioni del nostro classificatore su un singolo esempio, è necessario definire una **funzione di costo** (loss function). La scelta più naturale ed intuitiva nei problemi di classificazione è la *funzione costo 0-1*, che penalizza unicamente le predizioni errate:

$$l(\hat{y}, y) = \begin{cases} 0 & \text{se } \hat{y} = y \\ 1 & \text{se } \hat{y} \neq y \end{cases}$$

dove $\hat{y} = h(x)$ è la predizione del modello e y è l'etichetta reale.

A partire da questa funzione, definiamo due concetti fondamentali per valutare la bontà del modello:

- **Rischio Empirico (Errore di Training):** Calcola il tasso di errore sull'insieme di addestramento (Training Set) S. Corrisponde alla frazione di esempi che il modello classifica in modo errato:

$$RE_S(h) = \frac{1}{m} \sum_{i=1}^{m} l(\hat{y}^{(i)}, y^{(i)}) = \frac{|\{i \in S : \hat{y}^{(i)} \neq y^{(i)}\}|}{|S|}$$

- **Rischio Atteso (Errore di Test / Inaccuratezza):** È la vera metrica di interesse. Per definizione di valore atteso, calcolare l'aspettativa di una variabile indicatrice (che vale 1 in caso di errore e 0 altrimenti) equivale a calcolare la probabilità che l'evento si verifichi. Quindi, il rischio atteso rappresenta l'esatta *probabilità* che una nuova osservazione non vista venga classificata erroneamente:

$$RA(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[l(\hat{y}, y)] = Pr_{(x,y)\sim\mathcal{D}}[h(x) \neq y]$$

28 Il Metodo K-Nearest Neighbor (K-NN)

Uno degli approcci più semplici e potenti alla classificazione è l'algoritmo K-Nearest Neighbor. L'idea alla base è puramente geometrica: per classificare un nuovo punto x, l'algoritmo cerca i K punti del training set spazialmente più "vicini" ad esso. L'etichetta assegnata a x sarà semplicemente quella più frequente (la moda) tra i suoi K vicini. In caso di classificazione binaria, questo equivale a un voto a maggioranza.

28.1 La Nozione di Distanza

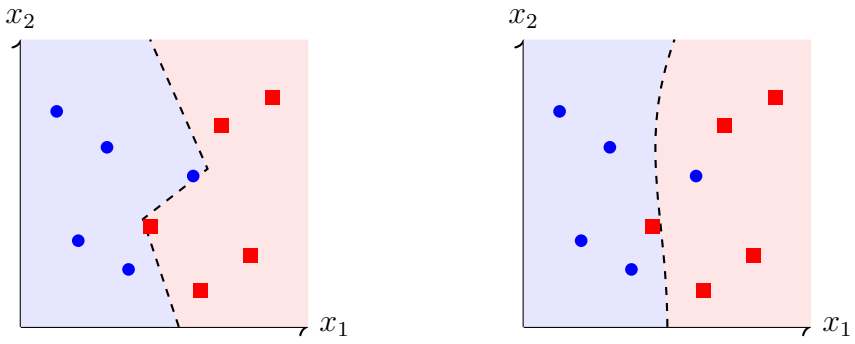
Il successo del K-NN dipende in modo critico dalla metrica utilizzata per definire la "vicinanza" nello spazio delle feature $\mathcal{X}$. Nello spazio vettoriale standard $\mathbb{R}^d$, la metrica predefinita è la **distanza euclidea**:

$$||x - x'|| = \sqrt{\sum_k (x_k - x'_k)^2}$$

Tuttavia, l'uso della distanza euclidea elementare può rivelarsi problematico in determinati domini. Per esempio, nel riconoscimento di immagini (come il dataset MNIST per cifre scritte a mano), la distanza euclidea pura sui pixel grezzi non è invariante per piccole traslazioni o rotazioni. Lo scivolamento di un'immagine di un solo pixel può causare un drastico aumento della distanza calcolata, sebbene l'oggetto rappresentato sia lo stesso. Per applicazioni avanzate, si rende quindi necessario lo studio e l'impiego di distanze più robuste e specializzate (come la *tangent distance* o lo *shape context*).

29 La Scelta di K: Il Compromesso Bias-Varianza

Il parametro K è un iperparametro cruciale che governa la complessità del modello e modella la **frontiera decisionale** (ovvero il confine geometrico che separa le diverse classi nello spazio).



K=1 (Overfitting)
Frontiera frastagliata

K grande (Generalizzazione)
Frontiera lisciata

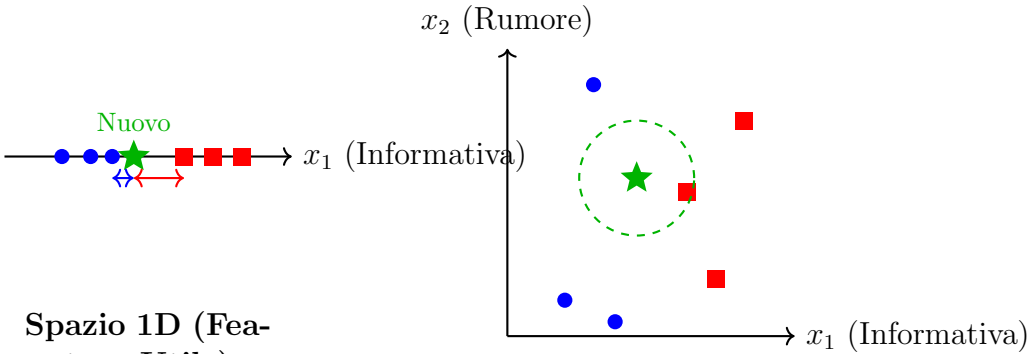
Figura 8: Confronto della frontiera decisionale al variare del parametro K. A sinistra, il modello si adatta al rumore (outlier); a destra, cattura il pattern globale.

- **K piccolo (es. K=1):** La frontiera decisionale diventa estremamente frastagliata e "insegue" ogni singolo punto del training set. Questo porta a un Rischio Empirico molto basso (per K=1 è dello 0%, in quanto ogni punto del training set è il vicino di se stesso), ma genera *overfitting*. Il modello diventa troppo sensibile al rumore statistico.
- **K grande:** La frontiera si ammorbidisce, inglobando zone più ampie a favore di una maggiore generalizzazione. Tuttavia, se K è troppo elevato, si rischia l'*underfitting*, ignorando pattern locali legittimi.

La scelta del valore ottimale di K deriva sempre da un accurato bilanciamento del **compromesso bias-varianza**, ottenuto tipicamente tramite tecniche di validazione incrociata.

30 Importanza della Feature Selection

Il K-NN è un algoritmo "pigro" (lazy learner) che assegna lo stesso peso matematico a tutte le dimensioni dello spazio degli input durante il calcolo della distanza. Di conseguenza, è particolarmente vulnerabile alla presenza di variabili rumorose o non informative.



Spazio 1D (Feature Utile)
Il punto verde è correttamente vicino alla classe blu.

Spazio 2D (Feature + Rumore)
Il rumore su x_2 altera la distanza: il vicino più prossimo diventa rosso.

Figura 9: Impatto delle variabili irrilevanti. L'aggiunta di una dimensione rumorosa espande lo spazio ortogonalmente, compromettendo il calcolo delle reali affinità geometriche.

Una singola variabile irrilevante può stravolgere il calcolo della distanza, mascherando la prossimità su feature realmente discriminanti. Una *feature selection* rigorosa a monte è pertanto essenziale prima di applicare questo algoritmo.

31 Efficienza Computazionale

Un grosso limite del K-NN nella sua implementazione naive è il costo computazionale a tempo di inferenza. Per trovare i K punti più vicini, l'algoritmo deve calcolare la distanza

tra il nuovo punto e *tutti* gli m esempi del training set a d dimensioni, richiedendo un tempo dell'ordine di $\mathcal{O}(m \cdot d)$ per ogni singola predizione.

Per rendere l'algoritmo scalabile in contesti reali, è prassi ricorrere a speciali strutture dati che preprocessano lo spazio degli input per velocizzare la ricerca (frequentemente implementate in librerie come *scikit-learn*):

- **K-d trees**
- **Ball trees**
- **Locality Sensitive Hashing (LSH)**

32 Garanzie Teoriche e Maledizione della Dimensionalità

Malgrado la sua semplicità, il K-NN offre solide garanzie asintotiche. Sotto ipotesi generali, si dimostra che per il caso $K = 1$, il rischio atteso $RA(h_S^{NN})$ è delimitato superiormente da:

Teorema 32.1 (Prestazioni di 1-NN)**.**

$$\mathbb{E}_S[RA(h_S^{NN})] \leq 2 \cdot RA(h^*) + c\sqrt{d}m^{-1/(d+1)}$$

dove $RA(h^*)$ è il rischio atteso minimo teoricamente ottenibile (l'errore di Bayes) e c è una costante dipendente dalla distribuzione dei dati.

Questa formula esprime due concetti vitali:

1. Quando la mole di dati tende all'infinito ($m \to \infty$), il termine di errore dipendente dal campione si annulla e il rischio atteso di 1-NN al limite non supera **il doppio del miglior errore possibile**.
2. Il tasso con cui il secondo termine (la dipendenza da m) decade è proporzionale a $m^{-1/(d+1)}$. Come si può notare matematicamente, all'aumentare delle dimensioni d del problema, questo esponente tende a 0, rendendo la decrescita estremamente lenta.

Quest'ultimo fenomeno è una manifestazione diretta della **Maledizione della Dimensionalità** (Curse of Dimensionality): in spazi ad alta dimensionalità, la nozione geometrica di "vicinato" perde di efficacia, e per mantenere lo stesso livello di accuratezza statistica, la quantità di dati di addestramento richiesti cresce esponenzialmente rispetto al numero di feature.

33 Introduzione alla Classificazione Generativa

A differenza della regressione, dove lo spazio di output è continuo (es. $\mathbb{R}$), nei problemi di classificazione lo spazio di output è costituito da un insieme finito di etichette qualitative. I metodi generativi (a differenza di approcci memory-based come il K-Nearest Neighbors) cercano di costruire un modello statistico per ciascuna classe, stimando la distribuzione di probabilità in grado di generare gli esempi osservati.

Definizione 33.1 (Modello Generativo)**.** Un modello generativo stima le probabilità congiunte $P(x, y)$ o, equivalentemente, le distribuzioni condizionate $P(x|y)$ e le probabilità a priori $P(y)$. Questo permette, teoricamente, di generare nuovi esempi simulando la distribuzione appresa, da questo il nome.

33.1 Il Criterio Bayesiano e la Minimizzazione del Rischio

Quando riceviamo un nuovo punto x da classificare, l'obiettivo è capire da quale distribuzione sia stato generato. Per farlo, utilizziamo il **criterio Bayesiano**. Vogliamo calcolare la probabilità a posteriori $P(y|x)$, ovvero la probabilità che avendo osservato x, questo appartenga alla classe y. Per il Teorema di Bayes:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)} \tag{40}$$

Dove:

- $P(x|y)$ è la verosimiglianza (likelihood), ovvero la distribuzione dei dati all'interno della classe y.
- $P(y)$ è la probabilità a priori della classe y (ad esempio, la sua frequenza relativa nel dataset).
- $P(x)$ è la probabilità marginale del dato, che agisce come fattore di normalizzazione.

Poiché il denominatore $P(x)$ è indipendente dalla classe y, al fine di trovare la classe più probabile a posteriori è sufficiente massimizzare il numeratore $P(x|y)P(y)$, infatti per un dato esempio il denominatore sarà lo stesso al variare delle classi.

33.2 Ottimalità del Criterio Bayesiano e Rischio Atteso

La classificazione generativa risulta ottimale in termini di rischio atteso quando si adotta una funzione di costo $0-1$. Questa funzione penalizza unicamente le predizioni errate, assegnando un costo unitario agli errori e un costo nullo alle predizioni corrette:

$$l(\hat{y}, y) = \begin{cases} 0 & \text{se } \hat{y} = y \\ 1 & \text{se } \hat{y} \neq y \end{cases} \tag{41}$$

Il rischio atteso di un classificatore h è definito come il valore atteso del costo sugli esempi estratti dalla distribuzione dei dati. Condizionando tale rischio all'osservazione di un particolare input x, il rischio atteso condizionato diventa esattamente la probabilità che il modello commetta un errore su quel punto:

$$\mathbb{E}[l|x] = P(h(x) \neq y|x) \tag{42}$$

Poiché la somma delle probabilità deve essere pari a 1, possiamo esprimere questa probabilità di errore in funzione della probabilità di classificare correttamente l'esempio:

$$\mathbb{E}[l|x] = 1 - P(h(x) = y|x) \tag{43}$$

Se l'obiettivo è minimizzare il rischio atteso complessivo, è necessario rendere la quantità $1 - P(h(x) = y|x)$ il più piccola possibile per ogni x. Ciò equivale matematicamente a massimizzare $P(h(x) = y|x)$, portando direttamente alla regola decisionale del classificatore bayesiano, che seleziona sempre l'etichetta con la probabilità a posteriori massima:

$$h(x) = \arg\max_{\hat{y} \in \mathcal{Y}} P(y = \hat{y}|x) \tag{44}$$

33.3 Il Discriminante

Nella pratica, per classificare un nuovo punto x, si cerca la classe j che massimizza la quantità $\pi_j P_j(x)$, dove $\pi_j = P(y = j)$ e $P_j(x) = P(x|y = j)$. Per facilitare questo calcolo si definisce il **discriminante** $\delta_j(x)$ applicando il logaritmo a tale prodotto:

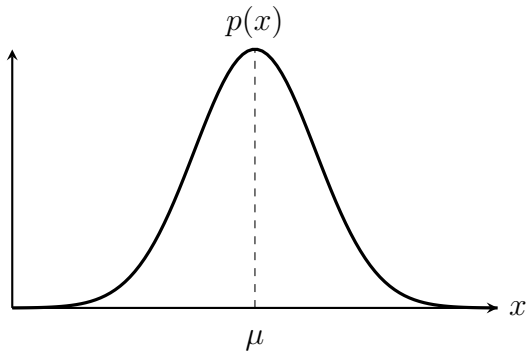
$$\delta_j(x) = \log(\pi_j P_j(x)) = \log(\pi_j) + \log(P_j(x)) \tag{45}$$

L'impiego del logaritmo è vantaggioso per due ragioni fondamentali. In primo luogo, essendo una funzione monotona crescente, preserva la posizione del massimo: l'etichetta che massimizza $\delta_j(x)$ è la stessa che massimizza la probabilità originale. In secondo luogo, il logaritmo trasforma i prodotti in somme; questo approccio risulta numericamente migliore poiché, lavorando con probabilità molto piccole, la somma contiene in modo significativo la perdita di precisione, evitando i tipici errori di underflow che si verificherebbero calcolando iterativamente il prodotto.

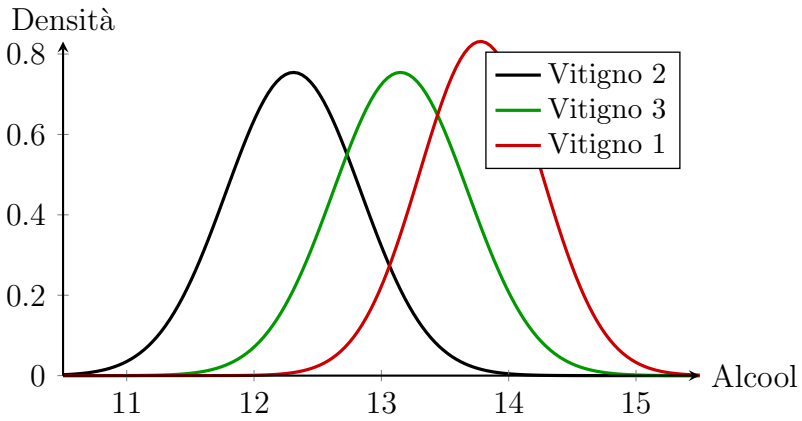
34 Analisi del Discriminante

34.1 Gaussiana Univariata

Supponendo che i dati abbiano una singola feature continua, si può assumere che la distribuzione $P(x|y)$ sia una gaussiana univariata.



Se si analizza un dataset utilizzando una singola feature (ad esempio, la concentrazione di alcol in un dataset contenente vini provenienti da tre cantine diverse), è possibile che le distribuzioni gaussiane stimate per le diverse classi risultino fortemente sovrapposte.



Questo fenomeno rappresenta un limite critico per il classificatore. Nei punti dello spazio in cui le curve di densità si intersecano o si sovrappongono pesantemente, i valori di verosimiglianza $P(x|y)$ diventano vicini per classi diverse. Di conseguenza, i valori della probabilità a posteriori calcolati con il teorema di Bayes (e i relativi discriminanti) risulteranno molto vicini tra loro, rendendo il modello incerto e incapace di discriminare a dovere gli esempi.

In queste ampie regioni di sovrapposizione, la decisione del classificatore finisce per dipendere quasi esclusivamente dalle probabilità a priori π_j, aumentando drasticamente il tasso di classificazione errata (che in un modello univariato può arrivare al 35%). Per risolvere questo problema, è necessario arricchire la rappresentazione introducendo ulteriori feature, permettendo così alle classi di separarsi in uno spazio a dimensionalità superiore passando a un modello multivariato.

34.2 Derivazione MLE per Gaussiana Univariata

Supponiamo che la distribuzione $P(x|y)$ segua una legge Normale $N(\mu, \sigma^2)$ (modello univariato, utile per studiare una singola feature come il grado alcolico dei vini). Come otteniamo i parametri empirici dai dati? Utilizziamo il Principio di Massima Verosimiglianza (MLE).

La densità di probabilità è:

$$P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{46}$$

Dato un dataset di m campioni $x^{(1)}, \ldots, x^{(m)}$, la funzione di verosimiglianza L è la produttoria delle probabilità marginali. Passando alla log-verosimiglianza $\log L$:

$$\begin{aligned} \log L(\mu, \sigma^2) &= \sum_{i=1}^{m} \log P(x^{(i)}) \\ &= -\frac{m}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{m}(x^{(i)} - \mu)^2 \end{aligned} \tag{47}$$

Massimizzare $\log L$ equivale a minimizzare $-\log L$. Derivando rispetto a μ e uguagliando a zero:

$$\frac{\partial}{\partial \mu}(-\log L) = \frac{1}{\sigma^2}\sum_{i=1}^{m}(x^{(i)} - \mu) = 0 \quad \Longrightarrow \quad \hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \tag{48}$$

Similmente, definendo $t = \sigma^2$ e minimizzando rispetto a t, si ottiene la varianza campionaria:

$$\hat{\sigma}^2 = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \hat{\mu})^2 \tag{49}$$

34.3 Gaussiana Multivariata

Nel caso di d feature è necessario l'utilizzo di una Gaussiana Multivariata, dove la varianza scalare σ^2 viene sostituita dalla **matrice di covarianza** $\Sigma \in \mathbb{R}^{d\times d}$, e la media aritmetica diventa il **vettore delle medie** $\mu \in \mathbb{R}^d$. In uno spazio a d dimensioni, dove le variabili

aleatorie $x_1, x_2, \ldots, x_d$ rappresentano le diverse feature dell'input, il vettore delle medie $\mu \in \mathbb{R}^d$ e la matrice di covarianza $\Sigma \in \mathbb{R}^{d \times d}$ sono definiti affiancati come segue:

$$\mu = \begin{bmatrix} \mathbb{E}[x_1] \\ \mathbb{E}[x_2] \\ \vdots \\ \mathbb{E}[x_d] \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} \mathrm{Var}(x_1) & \mathrm{Cov}(x_1, x_2) & \ldots & \mathrm{Cov}(x_1, x_d) \\ \mathrm{Cov}(x_2, x_1) & \mathrm{Var}(x_2) & \ldots & \mathrm{Cov}(x_2, x_d) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(x_d, x_1) & \mathrm{Cov}(x_d, x_2) & \ldots & \mathrm{Var}(x_d) \end{bmatrix} \tag{50}$$

La formula della densità di probabilità per la gaussiana multivariata è data da:

$$p(x) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right) \tag{51}$$

Vista la forma nel caso in cui fossero presenti molte features, la matrice di coviarianza diventa difficle da gestire.

34.4 QDA e LDA

La matrice di covarianza divide l'analisi del discriminante in due varianti a seconda della distribuzione delle variabili:

- **QDA (Quadratic Discriminant Analysis)**: Stima una matrice di covarianza Σ_j diversa per ogni classe. Applicando il logaritmo alla densità gaussiana, il discriminante diventa:

$$\delta_j(x) = \log(\pi_j) - \frac{1}{2}\log|\Sigma_j| - \frac{1}{2}(x-\mu_j)^\top \Sigma_j^{-1}(x-\mu_j) \tag{52}$$

  Sviluppando il prodotto, il termine quadratico in x risulta essere $-\frac{1}{2}x^\top \Sigma_j^{-1} x$. Poiché la matrice Σ_j^{-1} dipende strettamente dalla classe j, questo termine non può essere semplificato nel confronto tra classi, rendendo i confini di decisione delle curve quadratiche.

- **LDA (Linear Discriminant Analysis)**: Assume che tutte le classi condividano la medesima matrice di covarianza Σ. Sostituendo Σ_j con Σ ed espandendo il prodotto, si ottiene:

$$\delta_j(x) = \log(\pi_j) - \frac{1}{2}x^\top \Sigma^{-1} x + x^\top \Sigma^{-1}\mu_j - \frac{1}{2}\mu_j^\top \Sigma^{-1}\mu_j \tag{53}$$

  In questo caso, il termine quadratico in x è $-\frac{1}{2}x^\top \Sigma^{-1} x$. Essendo identico per tutte le classi (non dipende da j), esso si comporta come una costante rispetto al confronto e viene eliso. Il discriminante si riduce a un'equazione in cui x compare solo linearmente ($x^\top \Sigma^{-1}\mu_j$), rendendo lineari le frontiere di decisione.

Come si vede dalla figura(10) nella riga in altro, i dati hanno più o meno la stessa varianza e covarianza, quindi l'assunzione che la matrice di covarianza sia la stessa è lecita e infatti LDA funziona bene. Nella seconda riga viene meno questa premessa e quindi QDA classifica meglio.

Nella pratica la scelta tra i due metodi risiede in quale delle assunzioni che si fanno è più vicina alla realtà.

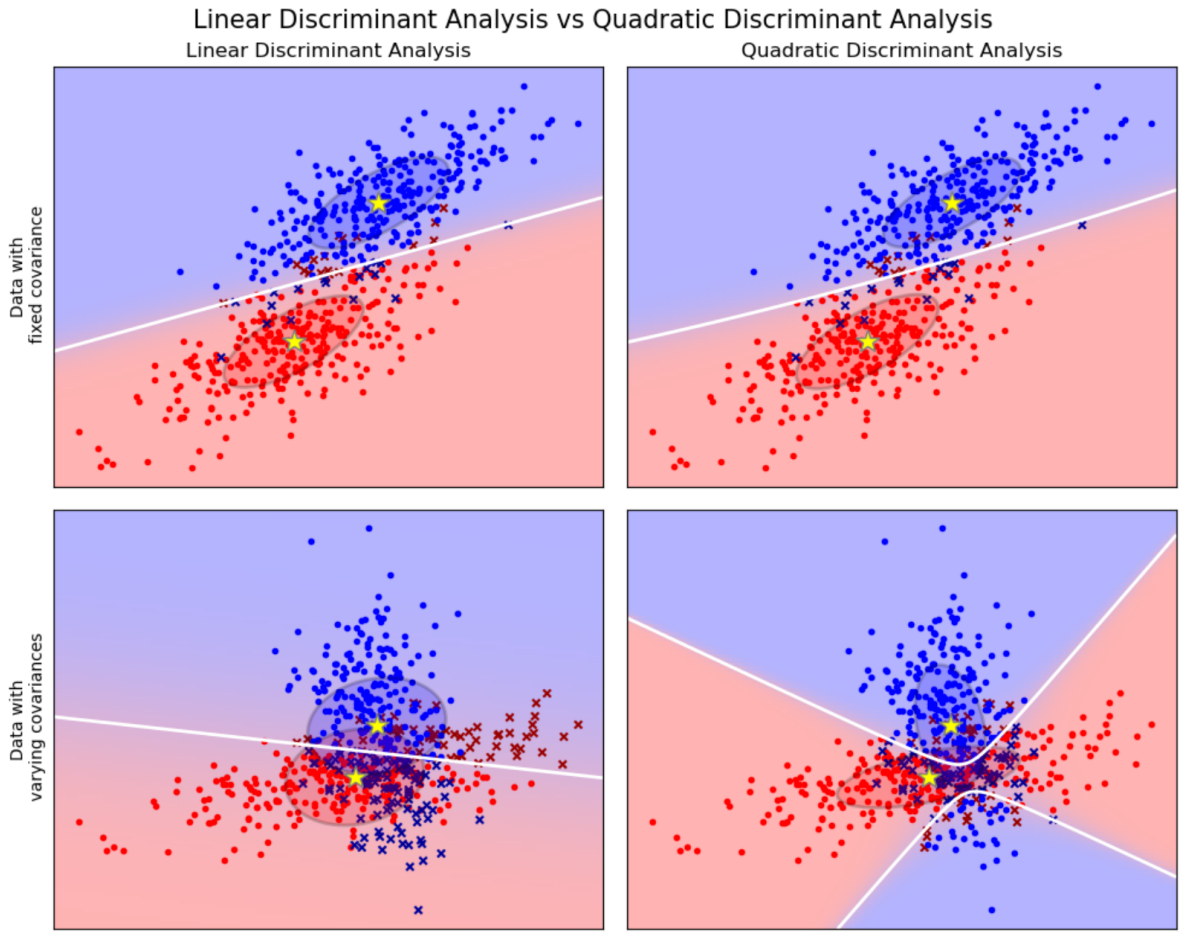


Figura 10: Confronto di classificazione tra Linear Discriminant Analysis (LDA) e Quadratic Discriminant Analysis (QDA) su dataset con covarianza fissa e variabile.

35 Classificatore Naive Bayes

Con un gran numero di feature (alta dimensionalità), il calcolo delle matrici di covarianza (necessario per QDA) diventa un collo di bottiglia inaccettabile (matrici $d \times d$).

Il **Naive Bayes** risolve il problema ponendo un'ipotesi molto forte: **si assumono le variabili in input statisticamente indipendenti data la classe**. Graficamente, questo corrisponde a considerare solo la diagonale principale delle matrici di covarianza. Si ottiene la probabilità congiunta come prodotto delle probabilità marginali:

$$P(\mathbf{x}|y) = \prod_{k=1}^{d} P(x_k|y) \tag{54}$$

Nonostante l'assunzione sia irrealistica nella realtà (le feature sono raramente indipendenti), dal punto di vista *classificatorio* il metodo si dimostra sorprendentemente solido, specie in ambiti come l'analisi testuale.

35.1 Esempio: Classificazione di Testi (Spam / No Spam)

Consideriamo il problema di classificare messaggi di testo definendo un dizionario D composto da d parole (ad esempio, $D = \{\text{'offerta'}, \text{'viaggio'}, \text{'affare'}, \dots\}$). Ogni messaggio viene codificato tramite un vettore binario $x \in \{0,1\}^d$, in cui l'elemento $x_k = 1$ se e solo se la k-esima parola del dizionario è presente nel testo, e $x_k = 0$ altrimenti. Le etichette di classe sono $y \in \{1(\text{Spam}), 0(\text{No Spam})\}$.

Un modello categorico generale che stimi senza approssimazioni la distribuzione $P(x|y)$ richiederebbe ben $2^d - 1$ parametri per ciascuna classe. Per un dizionario di dimensioni realistiche, questo numero cresce esponenzialmente ed è computazionalmente inattuabile.

Adottando l'assunzione di indipendenza del **Naive Bayes**, la probabilità del vettore x data la classe y si fattorizza come:

$$P(x_1, \dots, x_d|y) = \prod_{k=1}^{d} P(x_k|y) \tag{55}$$

In questo modo, il numero di parametri necessari crolla a una dipendenza lineare rispetto a d, richiedendo complessivamente solo $2d + 1$ parametri:

- $\phi_{k|y=1} = P(x_k = 1|y = 1)$: probabilità che la k-esima parola del dizionario appaia in un messaggio Spam.
- $\phi_{k|y=0} = P(x_k = 1|y = 0)$: probabilità che la k-esima parola del dizionario appaia in un messaggio No Spam.
- Probabilità a priori delle classi: $\pi_1 = P(y = 1)$ e $\pi_0 = 1 - \pi_1$.

La stima di questi parametri si ottiene facilmente tramite il principio di Massima Verosimiglianza (MLE), calcolando le frequenze relative sul dataset di training:

$$\hat{\phi}_{k|y=1} = \frac{|\{i : x_k^{(i)} = 1, y^{(i)} = 1\}|}{|\{i : y^{(i)} = 1\}|} \tag{56}$$

Questa formula corrisponde empiricamente al conteggio dei messaggi Spam che contengono la k-esima parola, diviso per il numero totale di messaggi Spam.

35.1.1 Esempio: Classificazione Testuale (Filtro Spam)

Immaginiamo di definire un dizionario $D = \{\text{'offerta'}, \text{'viaggio'}, \text{'affare'}, \dots\}$ di dimensione d. Ogni messaggio viene codificato come un vettore binario $x \in \{0,1\}^d$, dove 1 indica la presenza della parola nel testo. Le etichette di output sono $y \in \{0(\text{No Spam}), 1(\text{Spam})\}$. Se volessimo stimare il modello normale, per d parole servirebbero $2^d - 1$ parametri: un numero computazionalmente irraggiungibile.

Con il Naive Bayes, servono solo:

- La probabilità che l'i-esima parola appaia in un messaggio di Spam: $\phi_{k|y=1}$
- La probabilità che l'i-esima parola appaia in un messaggio Normale: $\phi_{k|y=0}$
- La probabilità a priori della classe: $\pi_1 = P(y=1)$ e $\pi_0 = 1 - \pi_1$

Otteniamo quindi un numero di parametri lineare rispetto a d. La stima avviene in maniera banale contato le frequenze:

$$\hat{\phi}_{k|y=1} = \frac{\text{Numero di occorrenze della parola } k \text{ in messaggi Spam}}{\text{Totale dei messaggi Spam}} \tag{57}$$

35.2 L'uso dei logaritmi e l'Underflow Numerico

Per classificare un testo x, il punteggio Naive Bayes comporta la moltiplicazione di molte probabilità inferiori a 1:

$$P(y=1|x) \propto \hat{\pi}_1 \cdot \hat{\phi}_{1|y=1} \cdot \hat{\phi}_{2|y=1} \dots \tag{58}$$

Lavorare con produttorie così lunghe di numeri minuscoli genera il problema dell'**underflow**, portando l'elaboratore a troncare i valori a 0. Come visto per il criterio Bayesiano generale, la soluzione è l'uso dei logaritmi:

$$\log(\hat{\pi}_1 \cdot \hat{\phi}_{1|y=1} \cdot \hat{\phi}_{3|y=1}) = \log \hat{\pi}_1 + \log \hat{\phi}_{1|y=1} + \log \hat{\phi}_{3|y=1} \tag{59}$$

Trasformando il prodotto in somma, la perdita di precisione è mantenuta entro limiti gestibili, garantendo un calcolo affidabile.

35.3 I "Cigni Neri" e la Regolarizzazione di Laplace

Esiste un'anomalia nel calcolo frequentista standard: cosa succede se una parola perfettamente lecita non compare mai, per puro caso, all'interno del training set per la classe "No Spam"? La formula le assegnerà probabilità 0. In fase di inferenza, se questa parola compare in un testo altrimenti palesemente sano, la produttoria (e il logaritmo) si annulleranno istantaneamente forzando il modello all'errore.

Per mitigare questo problema si usa uno **smoothing parametrico** (Laplace Smoothing) tramite un fattore α:

$$\phi_{k|y=1} = \frac{\alpha + \text{occorrenze}_{k,y=1}}{2\alpha + \text{totale}_{y=1}} \tag{60}$$

Con $\alpha \to \infty$, la probabilità tende al 50% (massima incertezza). Modulando α, si impone un minimo *prior* non nullo anche a parole sconosciute, rendendo il classificatore Naive Bayes molto più resiliente.

36 Stima di Massima Verosimiglianza (MLE)

I parametri delle distribuzioni vengono trovati tramite il principio di Massima Verosimiglianza. Ad esempio, per una feature binaria (come la presenza di una parola) si usa la distribuzione di Bernoulli.

Siano $x^{(1)}, \ldots, x^{(m)} \in \{0, 1\}$ esempi generati da una Bernoulli con parametro θ. Vogliamo trovare il θ più plausibile:

$$L(\theta|x) = \prod_{i=1}^{m} P(x^{(i)}) = \prod_{i=1}^{m} \theta^{x^{(i)}} (1-\theta)^{1-x^{(i)}} \tag{61}$$

Per ragioni numeriche (ed evitare l'underflow generato dalla moltiplicazione di molte probabilità basse), si minimizza la log-verosimiglianza negativa:

$$-\log L(\theta|x) = -\sum_{i=1}^{m} \left[x^{(i)} \log(\theta) + (1 - x^{(i)}) \log(1-\theta) \right] \tag{62}$$

Annullando la derivata prima rispetto a θ, si ricava agilmente la formula dello stimatore:

$$\hat{\theta} = \frac{1}{m} \sum_{i=1}^{m} x^{(i)} \tag{63}$$